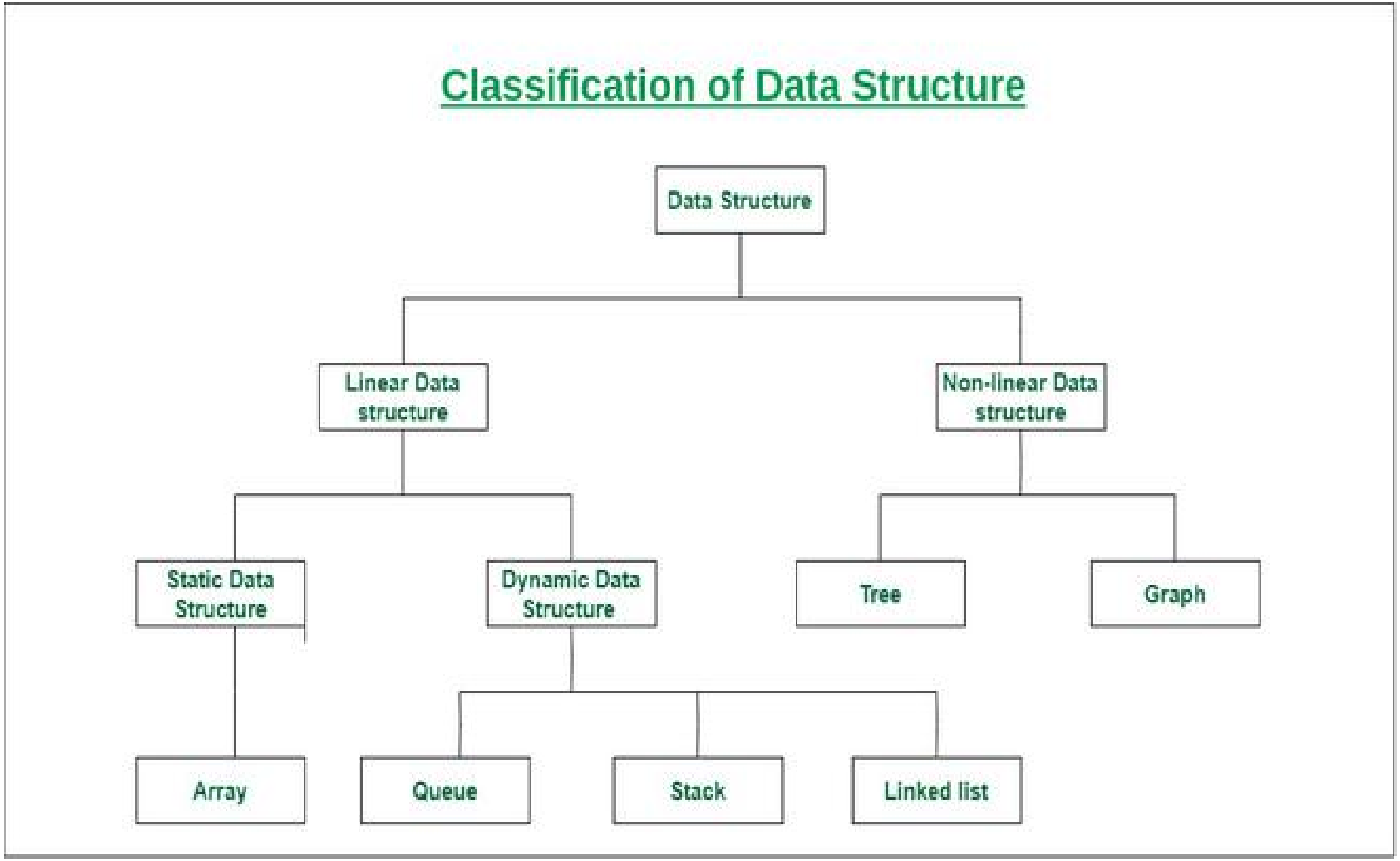
IT12 : Data Structure and Algorithms

What is Data Structure

- A data structure is a storage that is used to store and organize data. It is a way of arranging data on a computer so that it can be accessed and updated efficiently.
- A data structure is not only used for organizing the data. It is also used for processing, retrieving, and storing data.

What is Data Structure



What is Linked List

- A **linked list** is a linear data structure where elements (called **nodes**) are stored in a sequence, but not necessarily in contiguous memory locations.

Each node has two parts:

- **Data** – the actual value stored.
- **Pointer (or reference)** – a link to the next node in the sequence.
- Unlike arrays, linked lists don't need continuous memory blocks, so they allow **dynamic memory allocation**, **efficient insertion and deletion**, but **slower random access**.

- A **linked list** is like a chain of connected items where you can easily insert or remove elements without shifting the rest.

Real Life Examples of Linked List

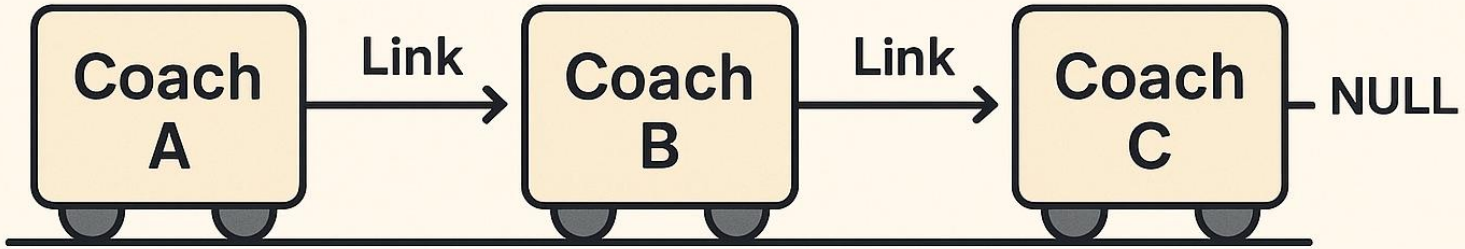
Example 1: Train Coaches

- Imagine a train: each coach (node) has passengers (data) and a connector (pointer) that links it to the next coach.
- If you want to add a new coach, you just connect it to the existing chain without rearranging all coaches.

- **Example 2: Playlist of Songs**
- Each song (node) contains the track (data) and a link to the next song (pointer).
- You can skip, add, or remove songs without shifting the entire playlist, just by changing links.

Diagram of Linked List

Linked List





Node (Coach)
Stores data (e.g. passenger list).
Has a pointer to the next coach.



Head (Engine)
First node of the linked list,
From the head, we can reach all other nodes.

NULL (End marker)
Last node's link points to NULL, meaning end of thelist.

Types of Linked List

Type	Structure	Use Case Example
Singly Linked List	Each node points to **next** node only.	Playlist of songs
Doubly Linked List	Each node has two pointers: **next** and **previous**.	Browser back/forward navigation
Circular Linked List	Last node points back to the **first node**.	Multiplayer games turn rotation

Operations on Linked List

Operation	Description
Traversal	Go through each node from head to end
Insertion	Add a new node at beginning, middle, or end
Deletion	Remove a node
Search	Find an element in the list

Algorithm of Linked List

Steps:

1. **Start**
2. Initialize head = NULL.
3. For each new node you want to add:
    1. Allocate memory for the new node.
    2. Store data in the new node.
    3. Set new_node.next = NULL.
4. If head == NULL (the list is empty):
    1. Make head = new_node (first node).
5. Else (if the list already has nodes):
    1. Traverse from head to the last node (where next == NULL).
    2. Set last.next = new_node.
6. Repeat step 3–5 until all nodes are created.
7. **Stop**

Add node at Beginning Linked List

1.Create a new node

Allocate memory for the new node.

Set new_node.data = value.

2.Link the new node to the current list

Set new_node.next = head (current first node).

3.Update the head

Set head = new_node.

4.Done

The new node is now the first node of the list.

Add node after position Linked List

Steps:

- **Create a new node**
  - Allocate memory for the new node.
  - Set new_node.data = value.
- **Traverse to the given position #** (while count < position and current != NULL)
  - Start from head.
  - Move forward position times to reach the node after which we insert.
  - Let this node be current.
- **Adjust the links**
  - Set new_node.next = current.next
  - Set current.next = new_node
- **Done**
  - Node inserted after the given position

Create Linked List


```
# Node class
class Node:
    def __init__(self, data):
        self.data = data  # store data
        self.next = None  # store reference
to next node

# Linked List class
class LinkedList:
    def __init__(self):
        self.head = None  # initially empty
list

    # Insert at the end
    def append(self, data):
        new_node = Node(data)
        if self.head is None:    # if list is
empty
            self.head = new_node
            return
        last = self.head
        while last.next:  # traverse to the
end
            last = last.next
        last.next = new_node
```

```
# Print the list
    def print_list(self):
        current = self.head
        while current:
            print(current.data, end=" -> ")
            current = current.next
        print("None")

# Usage
ll = LinkedList()
ll.append(10)
ll.append(20)
ll.append(30)
ll. print_list()
```

Add node at Begining


```
# Insert at the beginning
    def prepend(self, data):
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node
```

```
# Usage
ll = LinkedList()
ll.prepend(100)
```

Add node after given position


```
# Function to insert after a given position
    def insert_after(self, prev_data, new_data):
        temp = self.head
        while temp:
            if temp.data == prev_data:
                break
            temp = temp.next
        if temp is None:
            print(f"Node with data {prev_data} not found.")
            return
        new_node = Node(new_data)
        new_node.next = temp.next
        temp.next = new_node
```

```
# Usage
ll = LinkedList()
ll.insert_after(20, 25)  # insert 25 after node with data 20
```

Delete node


```
def delete_node(self, key):
    temp = self.head

    # Case 1: Head node itself is to be deleted
    if temp and temp.data == key:
        self.head = temp.next
        temp = None
        return

    # Case 2: Search for the key
    prev = None
    while temp and temp.data != key:
        prev = temp
        temp = temp.next

    # If key not found
    if temp is None:
        return

    # Unlink the node
    prev.next = temp.next
    temp = None
```

```
# Usage
ll = LinkedList()
ll.delete_node(20)   # Delete node with value 20

print("After deletion:")
ll.print_list()
```

What is Doubly Linked List

A Doubly Linked List (DLL) is a type of linked list where each node contains:

- Data
- Pointer to Next Node
- Pointer to Previous Node.

This allows **bidirectional traversal**:

- Forward from head to tail
- Backward from tail to head

Real Life examples

- Browser's Back/Forward history
- Undo/Redo feature in text editors
- Navigation menus (move next/previous)

Key Points

- DLL allows movement both ways.
- Slightly more memory needed (extra pointer).
- Insertion & deletion are faster than singly linked lists when node is known.

Advantages of DLL

- Can be traversed in both directions
- Easier deletion (no need to traverse from head to find previous node)
- More flexible compared to singly linked list

Disadvantages :

- Requires extra memory (extra pointer prev)
- Slightly complex implementation

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None
```

DLL Insertion at the End


```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

class DoublyLinkedList:
    def __init__(self):
        self.head = None

    def insert_at_end(self, data):
        new_node = Node(data)

        # If list is empty
        if self.head is None:
            self.head = new_node
            return
        # Otherwise, traverse to the end
        temp = self.head
        while temp.next:
            temp = temp.next

        temp.next = new_node
        new_node.prev = temp

    def display_forward(self):
        temp = self.head
        while temp:
            print(temp.data, end=" <-> ")
            temp = temp.next
        print("NULL")
```

DLL Insert after node


```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

class DoublyLinkedList:
    def __init__(self):
        self.head = None

def insert_after(self, target_data, new_data):
        temp = self.head
        while temp:
            if temp.data == target_data:
                new_node = Node(new_data)

                # Link new_node's next to current's next
                new_node.next = temp.next
                new_node.prev = temp
                # Adjust next node's prev pointer if it exists
                if temp.next:
                    temp.next.prev = new_node
                # Link current node to new node
                temp.next = new_node
                return
            temp = temp.next
        print(f"Element {target_data} not found in the list.")
```

What is Circular Linked List

- A **Circular Linked List** is a variation of a linked list in which:
- The **last node points back to the first node**.
- It can be **singly circular** (only next pointer loops back) or **doubly circular** (both next and prev form a circle)

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

Advantages of Circular Linked List

- Efficient use of memory (no NULL pointers)
- Can **continuously traverse** the list from any node
- Useful in **real-time applications** like scheduling

Disadvantages of Circular Linked List:

- More Complex Implementation
- Risk of Infinite Loop
- Harder to Debug
- Extra Care for Deletion

CLL Insertion at the End


```
class CircularLinkedList:
    def __init__(self):
        self.head = None

    def insert_at_end(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            new_node.next = self.head
          # point to itself
            return

temp = self.head
        while temp.next != self.head:   # reach
last node
            temp = temp.next

        temp.next = new_node
        new_node.next = self.head
```

```
def display(self):
        if not self.head:
            print("List is empty")
            return

        temp = self.head
        while True:
            print(temp.data, end=" -> ")
            temp = temp.next
            if temp == self.head:
                break
        print("(back to head)")
```

What is Stack

- A **Stack** is a linear data structure that follows the **LIFO (Last In, First Out)** principle:
- The **last element inserted** is the **first element removed**.
- Imagine a stack of plates 🍽️ → you put one on top and remove from the top.

Basic Operations

- **Push(x)** → insert an element at the top.
- **Pop()** → remove the top element.
- **Peek/Top()** → view the top element without removing it.
- **isEmpty()** → check if stack is empty.
- **isFull()** (in array-based stack) → check if no more elements can be added.

Real life Example

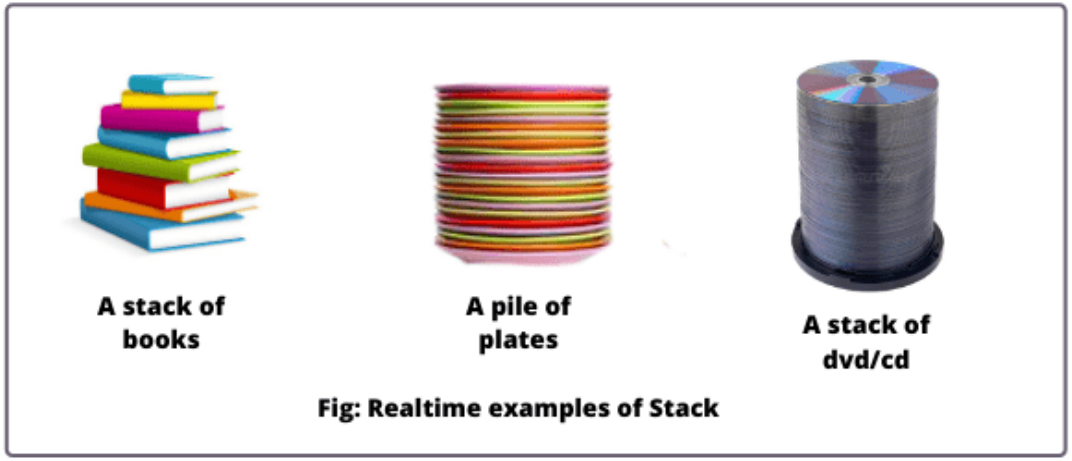
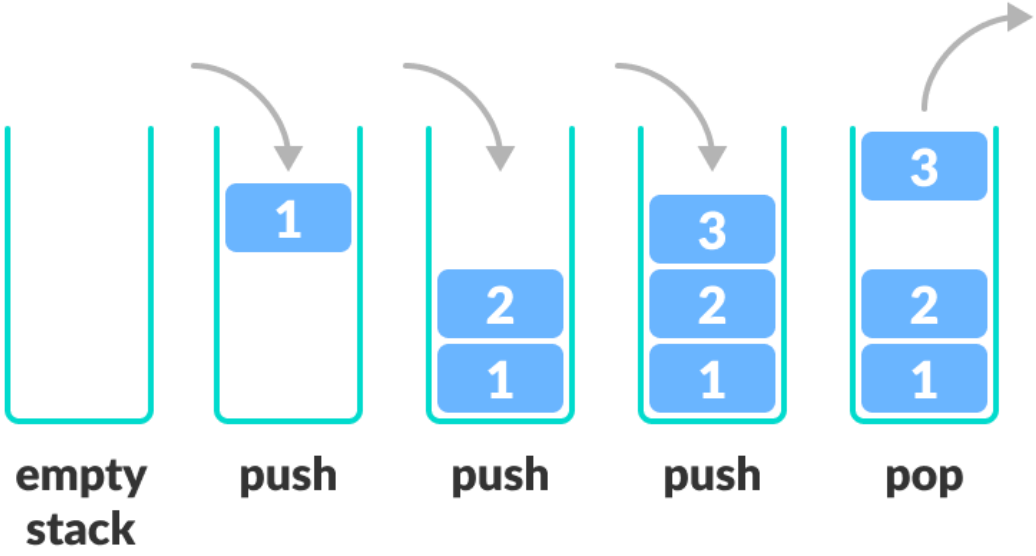
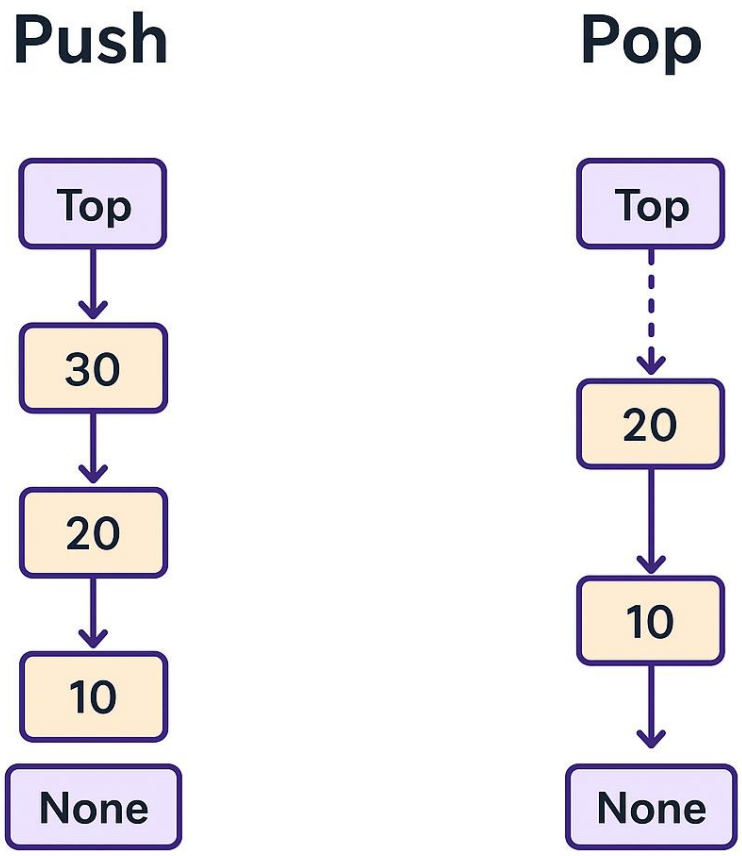


Fig: Realtime examples of Stack



Diagram



Stack Using Linked List


```
# Node class for linked list
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

# Stack using Linked List
class Stack:
    def __init__(self):
        self.top = None  # top of stack

    # Push element onto stack
    def push(self, data):
        new_node = Node(data)
        new_node.next = self.top
# link new node to current top
        self.top = new_node
        print(f"Pushed {data} onto stack")
```

```
# Pop element from stack
    def pop(self):
        if self.is_empty():
            print("Stack Underflow! No
elements to pop.")
            return None
        popped = self.top.data
        self.top = self.top.next
        print(f"Popped {popped} from
stack")
        return popped
```

Stack Using Linked List


```
def display(self):
        if self.is_empty():
            print("Stack is empty.")
            return
        temp = self.top
        print("Stack elements (top to
bottom): ", end="")
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")
```

```
stack = Stack()
stack.push(10)
stack.push(20)
stack.push(30)

stack.display()

print("Top element is:", stack.peek())

stack.pop()
stack.display()
```

Applications of Stack

- **Expression evaluation** (infix → postfix, prefix)
- **Undo/Redo operations** in text editors
- **Backtracking algorithms** (e.g., maze solving, DFS)
- **Function calls** (call stack in programming languages)
- **Browser history** (Back button)

Advantages & Disadvantages

- **Advantages**
- Simple to implement
- Useful in recursive and LIFO-based problems
- **Disadvantages**
- Limited access (only top can be accessed)
- In array implementation → fixed size (overflow possible)

What is Queue

- A **Queue** is a linear data structure that follows the **FIFO (First In, First Out)** principle:
- The **first element inserted** is the **first element removed**.
- Imagine a queue at a ticket counter → first come, first served.

Basic Operations

- **Enqueue(x)** → insert an element at the **rear**.
- **Dequeue()** → remove an element from the **front**.
- **Front/Peek()** → view the front element without removing.
- **isEmpty()** → check if queue is empty.
- **isFull()** (for array-based queue) → check if no more elements can be added

Types of Queues

- **Simple Queue** – linear queue with single front & rear.
- **Circular Queue** – last element connects to the first for efficient space usage.
- **Priority** Queue – each element has a priority, higher priority dequeued first.
- **Deque (Double-ended Queue)** – insertion & deletion allowed at both ends.

Applications of Queues

- CPU Scheduling
- Print Spooling
- Customer Service Systems
- I/O Buffers
- Breadth-First Search (BFS) in Graphs
- Tasks scheduling
- Admission process
- Payment Process

Advantages of Queue

Maintains Order (FIFO)

•Elements are processed in the exact order they were added.

•**Useful for Scheduling**

•Ideal for CPU scheduling, printer queues, and task management systems.

•**Simple to Implement**

•Can be implemented using **arrays** or **linked lists**.

•**Dynamic in Linked List Implementation**

•No fixed size, unlike array-based queues, so memory is used efficiently.

•**Supports Multiple Variants**

•Circular queue, priority queue, and deque allow flexibility for different applications.

Disadvantages of Queue

Limited Access

Only the **front and rear** elements can be accessed directly.

Wasted Space in Linear Array Queue

After multiple dequeues, empty slots at the front are unused unless circular queue is implemented.

Slightly More Complex in Circular/Deque Variants

Circular queues and double-ended queues require extra logic for pointer management.

Sequential Access

To access a middle element, you may have to traverse from front, which is inefficient

Queue Using Linked List


```
# Node class for linked list
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

# Queue implementation using linked list
class Queue:
    def __init__(self):
        self.front = None
        self.rear = None

    # Enqueue operation (add element)
    def enqueue(self, data):
        new_node = Node(data)
        if self.rear is None:
  # If queue is empty, both front and rear are the same
            self.front = self.rear = new_node
            print(f"Enqueued {data} (first element)")
            return
        self.rear.next = new_node
        self.rear = new_node
        print(f"Enqueued {data}")
```

```
# Dequeue operation (remove element)
    def dequeue(self):
        if self.is_empty():
            print("Queue Underflow! No elements to dequeue.")
            return None
        temp = self.front
        self.front = temp.next

        # If front becomes None, queue is now empty
        if self.front is None:
            self.rear = None
        print(f"Dequeued {temp.data}")
        return temp.data
```

Queue Using Linked List


```
# Check if queue is empty
    def is_empty(self):
        return self.front is None


    # Display all elements
    def display(self):
        if self.is_empty():
            print("Queue is empty.")
            return
        temp = self.front
        print("Queue elements (front → rear): ", end="")
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")
```

```
# Example usage
q = Queue()
q.enqueue(10)
q.enqueue(20)
q.enqueue(30)
q.display()
q.dequeue()
q.display()
```

Next Greater Element Using Linked List

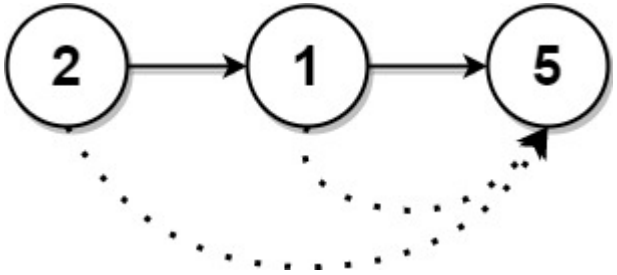
You are given the head of a linked list with n nodes.

For each node in the list, find the value of the **next greater node**. That is, for each node, find the value of the first node that is next to it and has a **strictly larger** value than it.

Return an integer array answer where answer[i] is the value of the next greater node of the ith node (**1-indexed**). If the ith node does not have a next greater node, set answer[i] = 0.

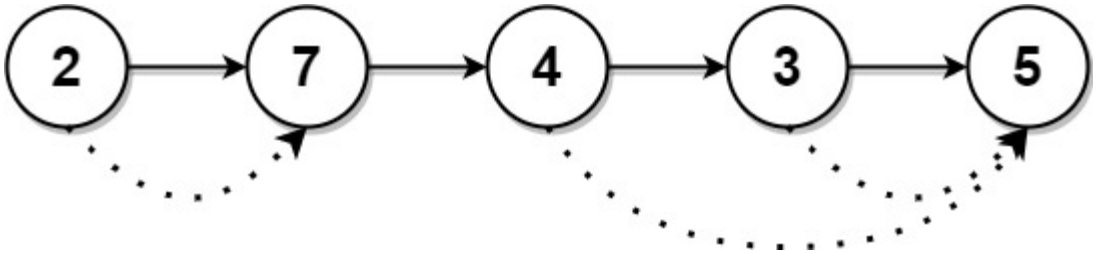
Next Greater Element Using Linked List

Example 1:



Input: head = [2,1,5] **Output:** [5,5,0]

Example 2:



Input: head = [2,7,4,3,5] **Output:** [7,0,5,5,0]

Next Greater Element Using Linked List


```
def next_greater_element_ll(ll):
    current = ll.head
    result = []

    # For each node, scan nodes to the right
    while current:
        next_node = current.next
        nge = -1

        while next_node:
            if next_node.data > current.data:
                nge = next_node.data
                break
            next_node = next_node.next

        result.append((current.data, nge))
        current = current.next

    return result
```

Tree :

A tree is a non-linear abstract data type with a hierarchy-based structure. It consists of nodes (where the data is stored) that are connected via links.

The tree data structure stems from a single node called a root node and has subtrees connected to the root.

The data structure is called a "tree" because it looks like a tree, only upside down

Drawback of Linear DS

Drawback	Applies to
Fixed size	Array, static Queue/Stack
Slow insertion/deletion	Array
No direct access (random)	Linked List, Stack, Queue
Linear search required	Array, List
Cannot represent complex data	All linear structures
Overflow/Underflow	Stack, Queue

Advantages of Non Linear DS

- **Hierarchical Data**: File systems, organizational models, etc.
- **Databases**: Used for quick data retrieval.
- **Routing Tables**: Used for routing data in network algorithms.
- **Sorting/Searching**: Used for sorting data and searching for data.
- **Priority Queues**: Priority queue data structures are commonly implemented using trees, such as binary heaps.

Real life Examples

a) File System

Your computer's folders → subfolders → files

Perfect example of a **tree structure**

b) Organization Hierarchy

CEO
→ Managers
→ Team Leads
→ Employees

Exactly like a **tree with parent–child relationships**

c) Family Tree

Grandparents
→ Parents
→ Children

Real life Examples

a) Social Networks

Facebook, Instagram

Each user = node

Connections/followers = edges

b) Google Maps / GPS Navigation

Locations = nodes

Roads = edges

Shortest path algorithms run on graphs

c) Airline Route Networks

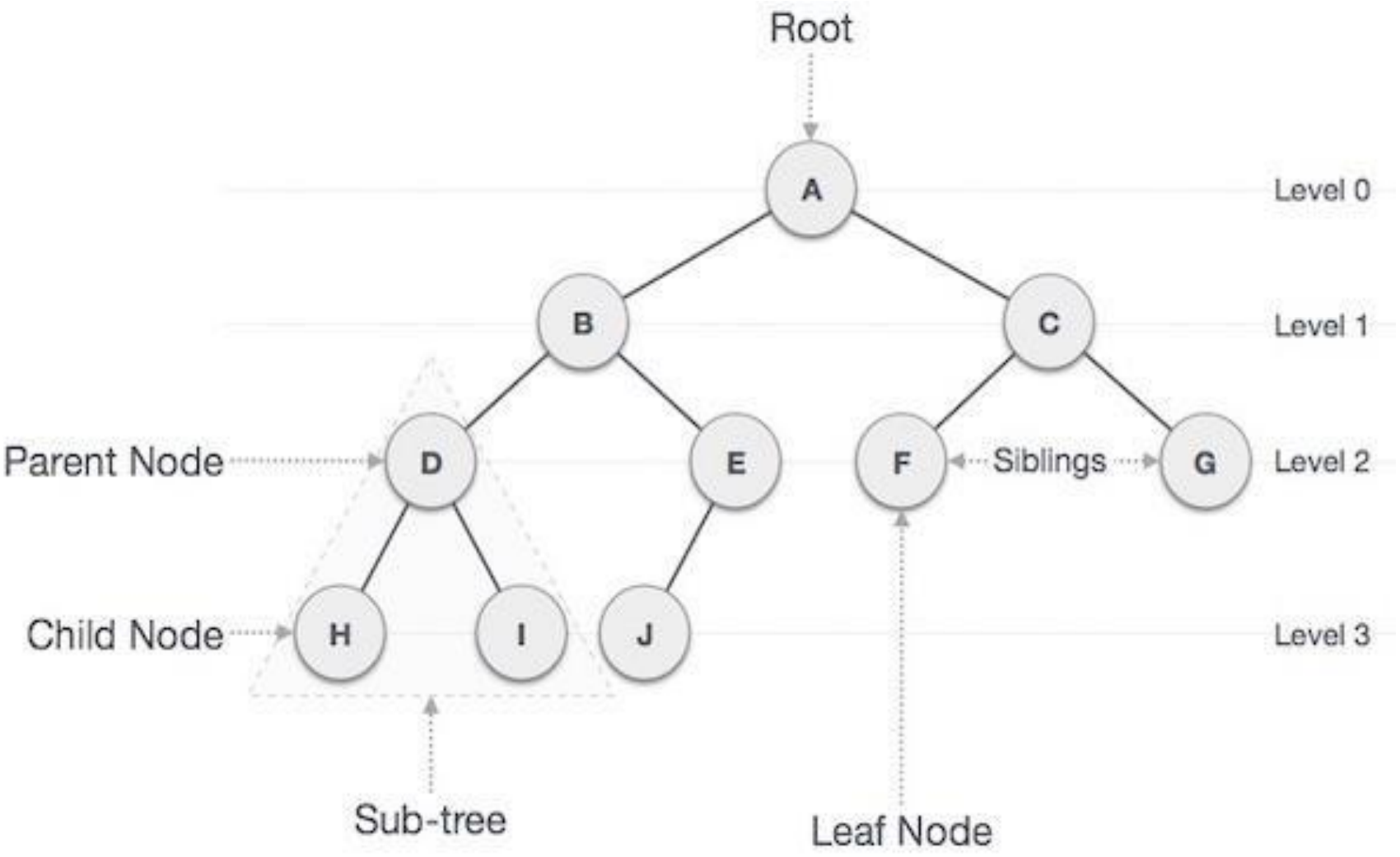
Airports = nodes

Flights = edges

d) Internet Network

Routers and servers connected → huge graph

Unit 2 : Tree Diagram



Unit 2 : Tree Terminologies

1. Node
A single element in the tree.
2. Root
The topmost node of the tree.
3. Edge
Connection between nodes.
4. Parent
Node that has children.
5. Child
Node directly connected below a parent.
6. Leaf Node
A node with **no children**.
7. Height
Length of the longest path from root to leaf.
8. Depth (Level)
Distance from root to the node.
9. Subtree
A tree entirely within another tree.

Types of Trees

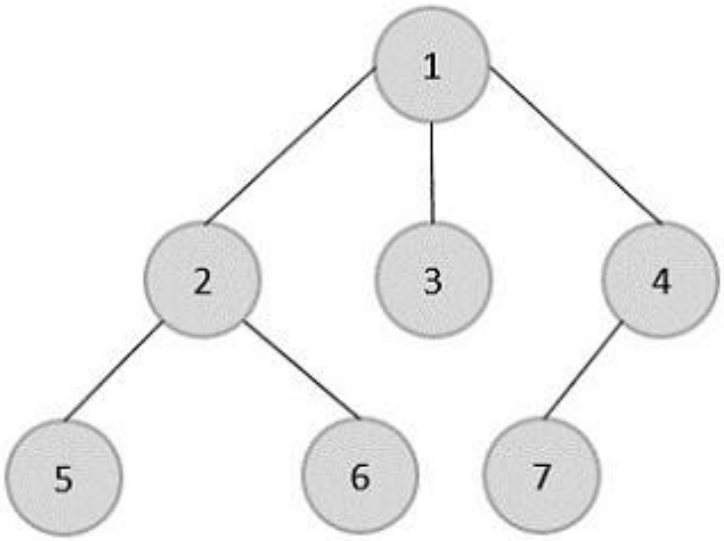
1. **General Trees**
2. **Binary Trees**
3. **Binary Search Trees**

General Trees

General trees are unordered tree data structures where the root node has minimum 0 or maximum n subtrees.

The General trees have no constraint placed on their hierarchy.

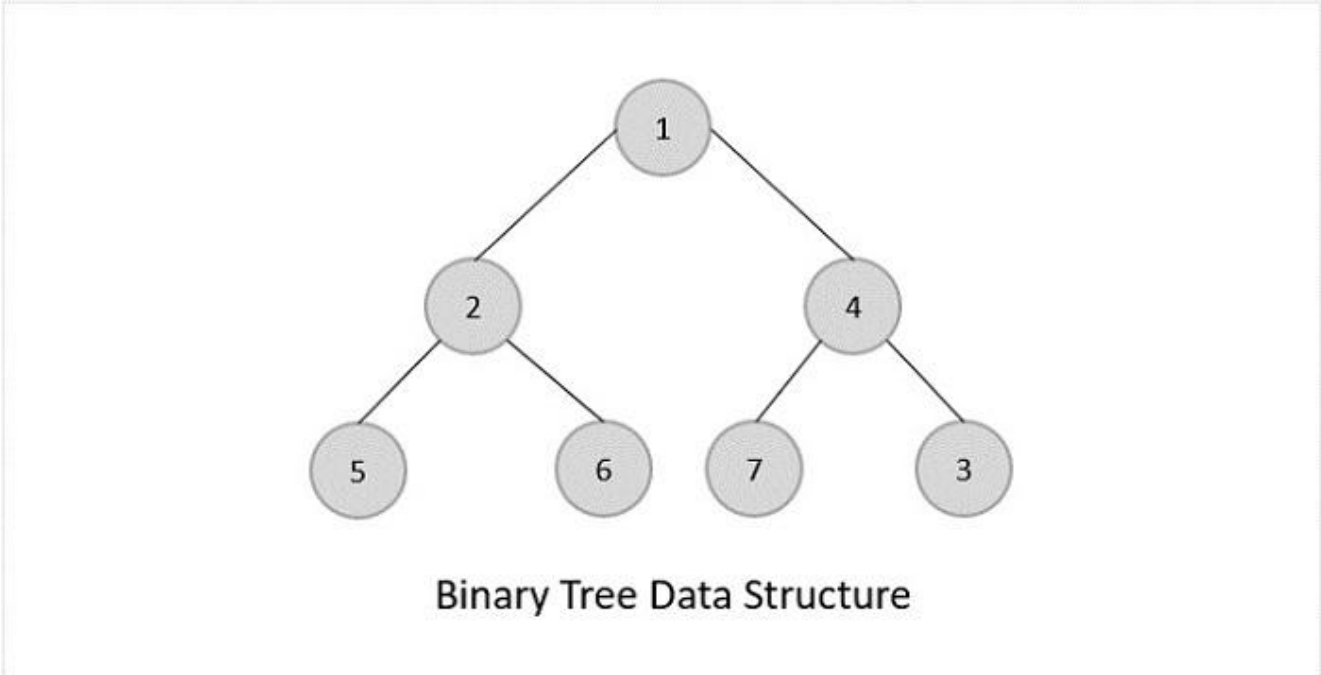
General Tree Structure



General Tree Data Structure

Binary Tree Structure

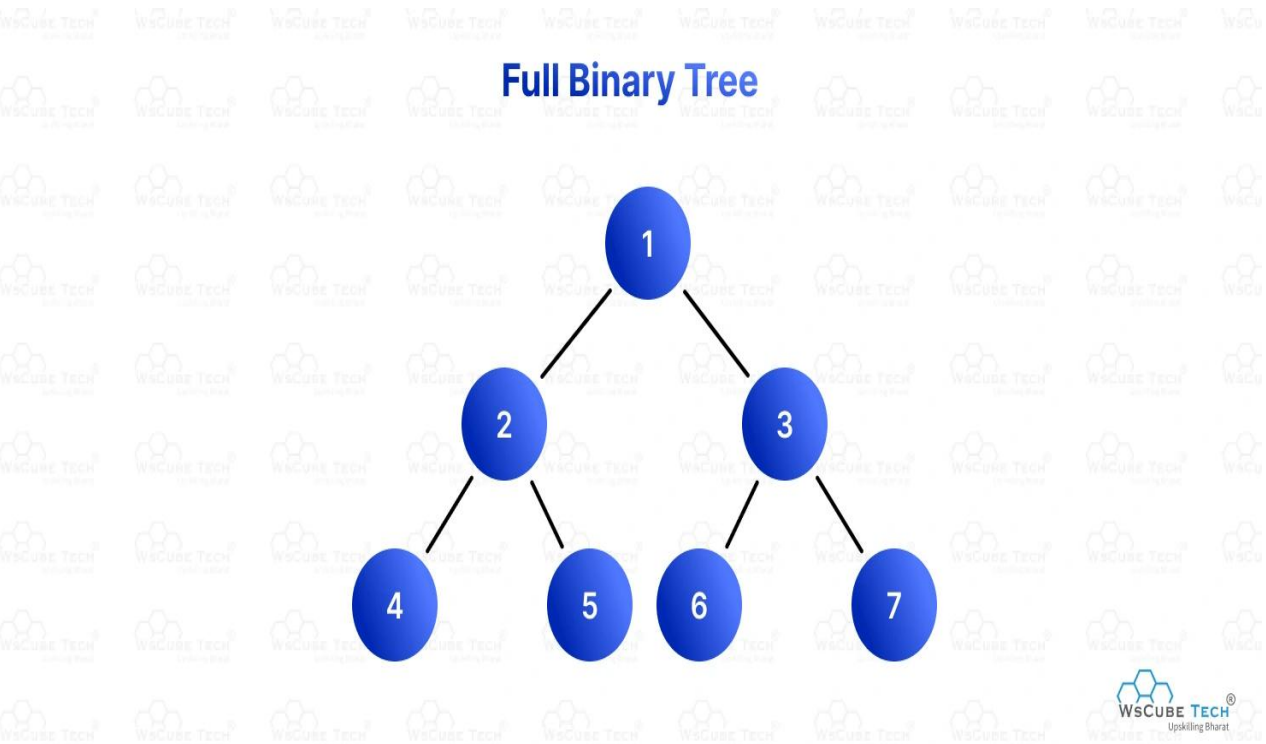
Binary Trees are general trees in which the root node can only hold up to maximum 2 subtrees: left subtree and right subtree. Based on the number of children, binary trees are divided into three types:



Binary Tree Data Structure

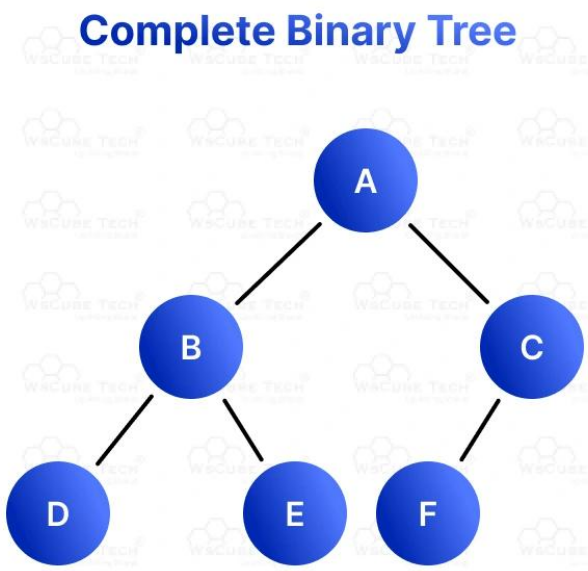
Binary Tree Structure

Full Binary Tree : A full binary tree is a tree where every node has either 0 or 2 children.



Binary Tree Structure

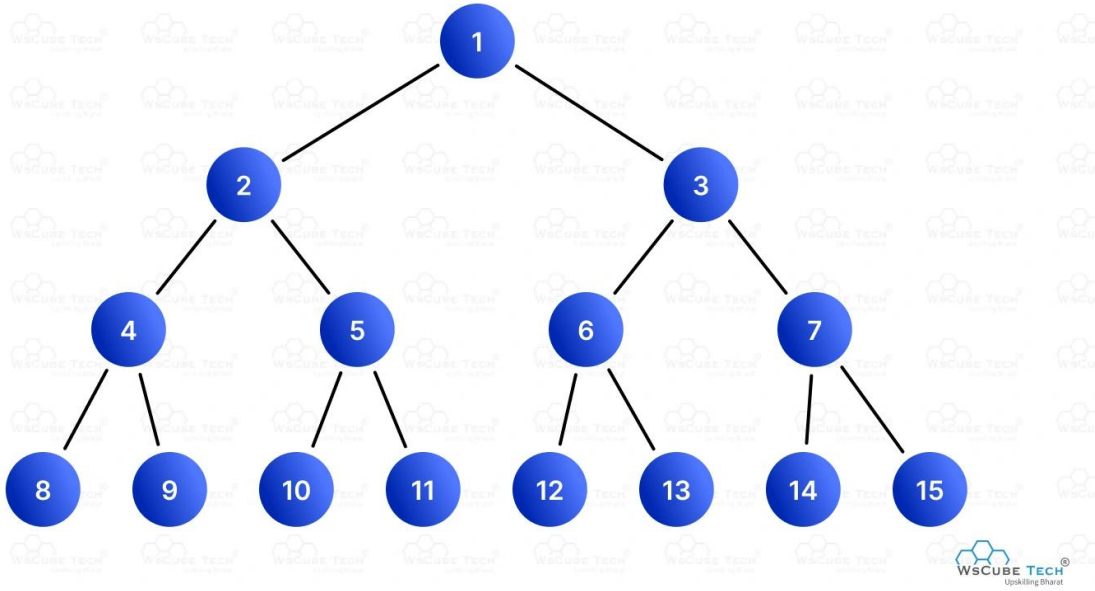
Complete Binary Tree: A complete binary tree is a tree where all levels are fully filled except possibly the last level, which is filled from left to right



WsCube Tech
Upskilling Bharat

Binary Tree Structure

Perfect Binary Tree: A perfect binary tree is a tree where all internal nodes have exactly two children and all leaf nodes are at the same level.



Tree Traversing Methods

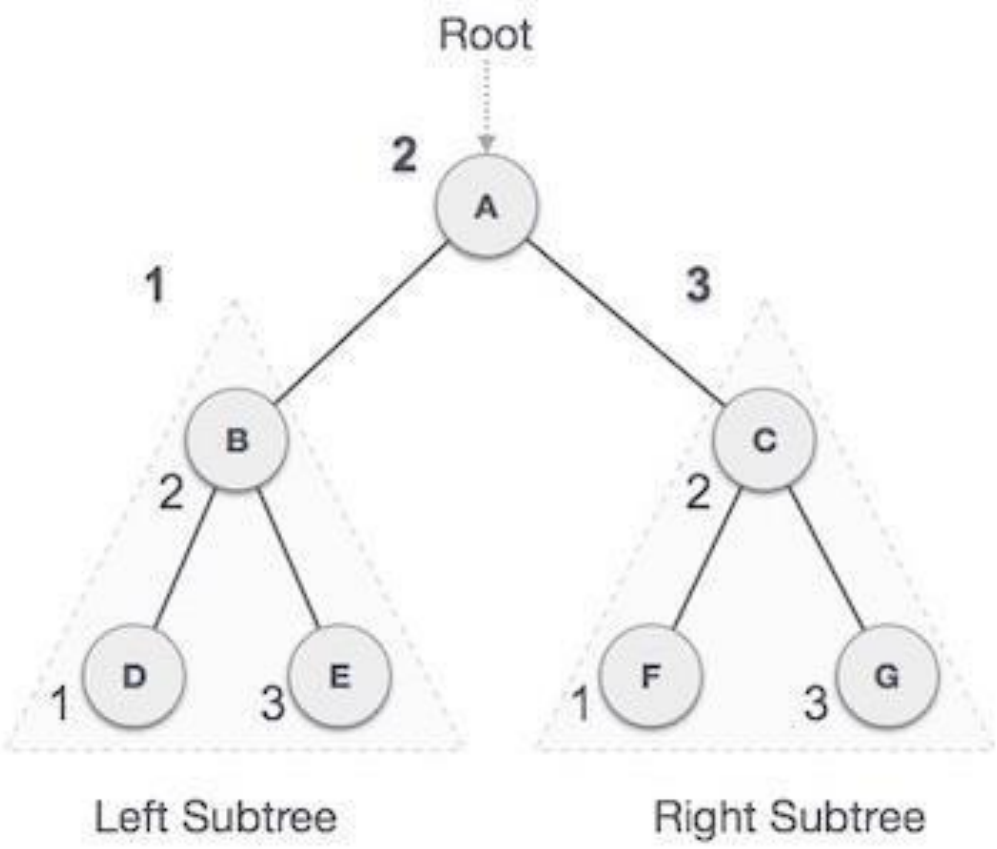
There are three ways which we use to traverse a tree :

- **In-order Traversal (Left-Root-Right) :**In this traversal method, the left subtree is visited first, then the root and later the right sub-tree.

- **Pre-order Traversal(Root-Left-Right):** In this traversal method, the root node is visited first, then the left subtree and finally the right subtree.

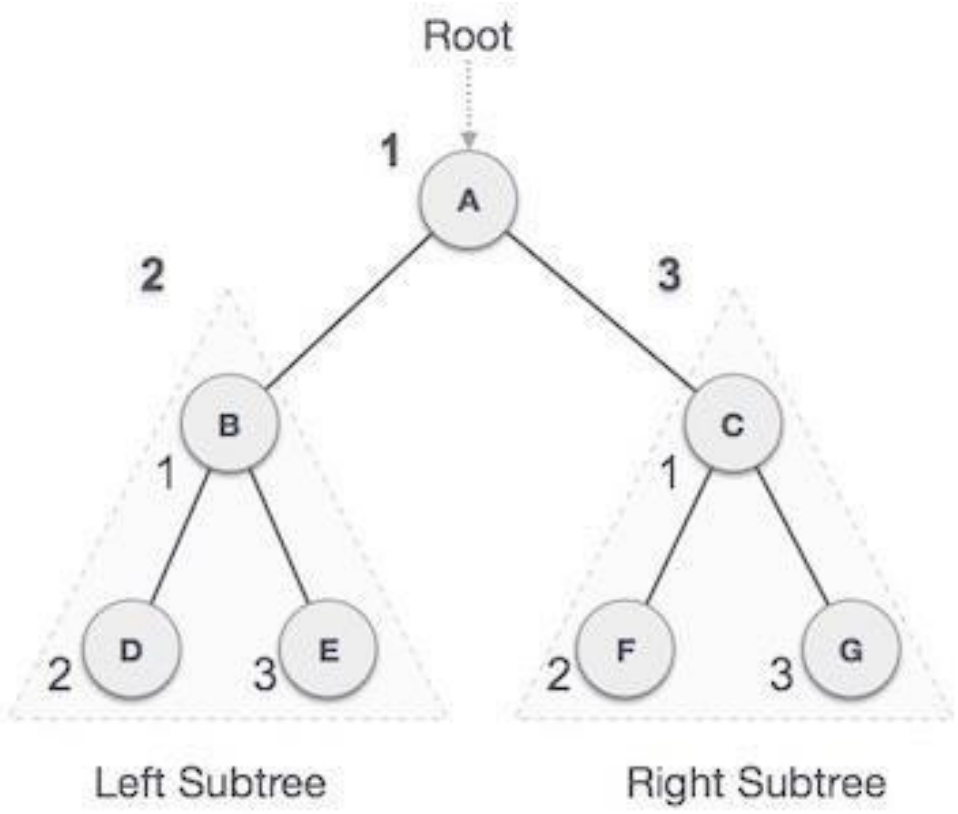
- **Post-order Traversal(Left-Right-Root):** In this traversal method, the root node is visited last, hence the name. First we traverse the left subtree, then the right subtree and finally the root node.

Inorder Traversing – Left –Root - Right



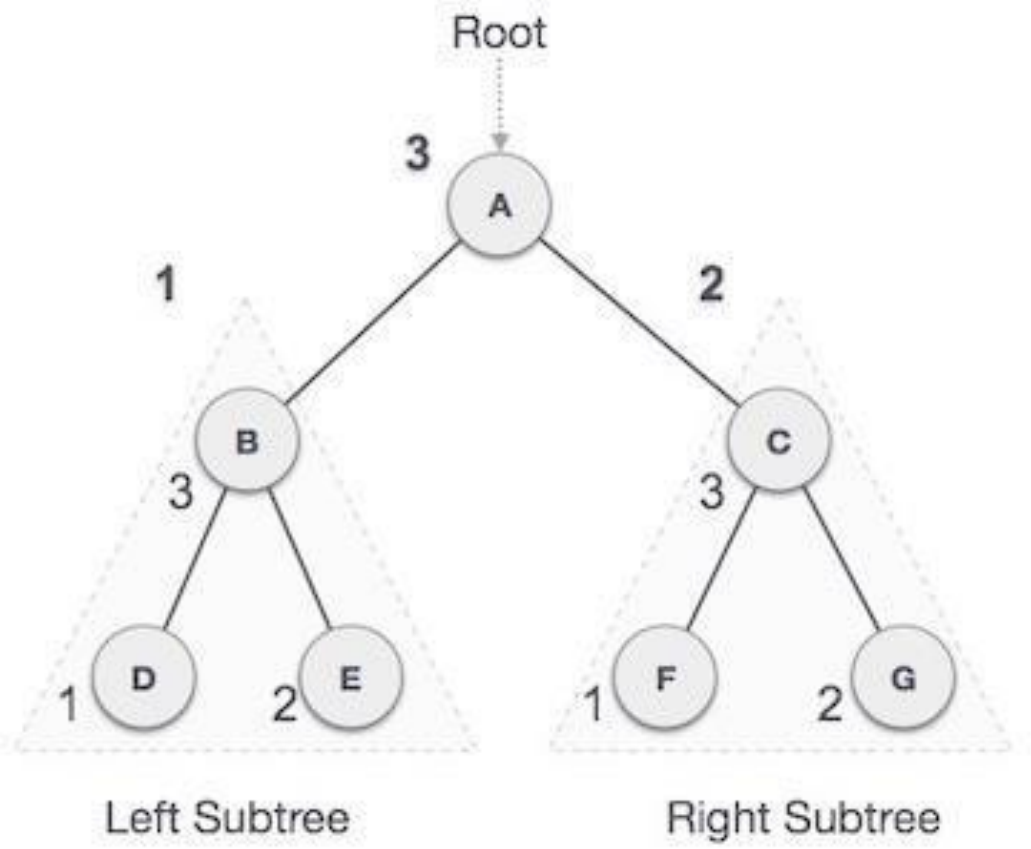
Output - D B E A F C G

Preorder Traversing – Root- Left - Right



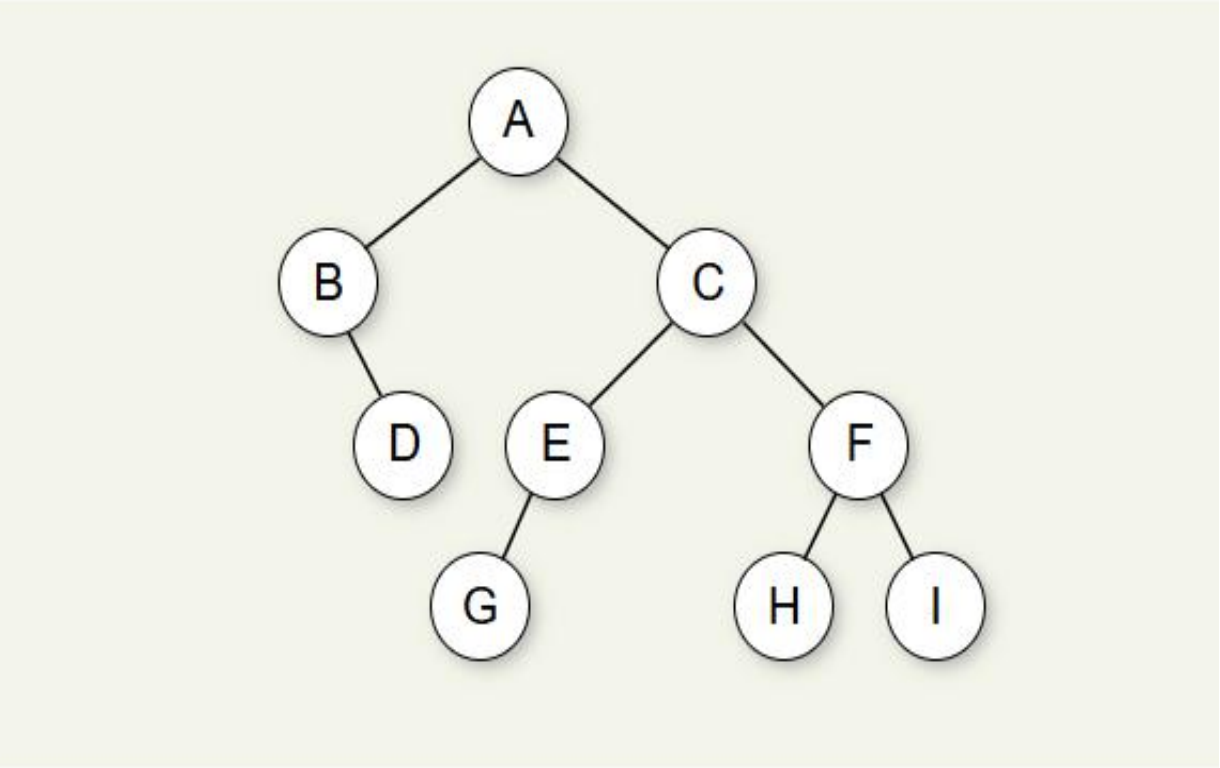
A → B → D → E → C → F → G

Postorder Traversing –Left – Right-Root



D → E → B → F → G → C → A

Find Inorder, Preorder and Postorder of below tree

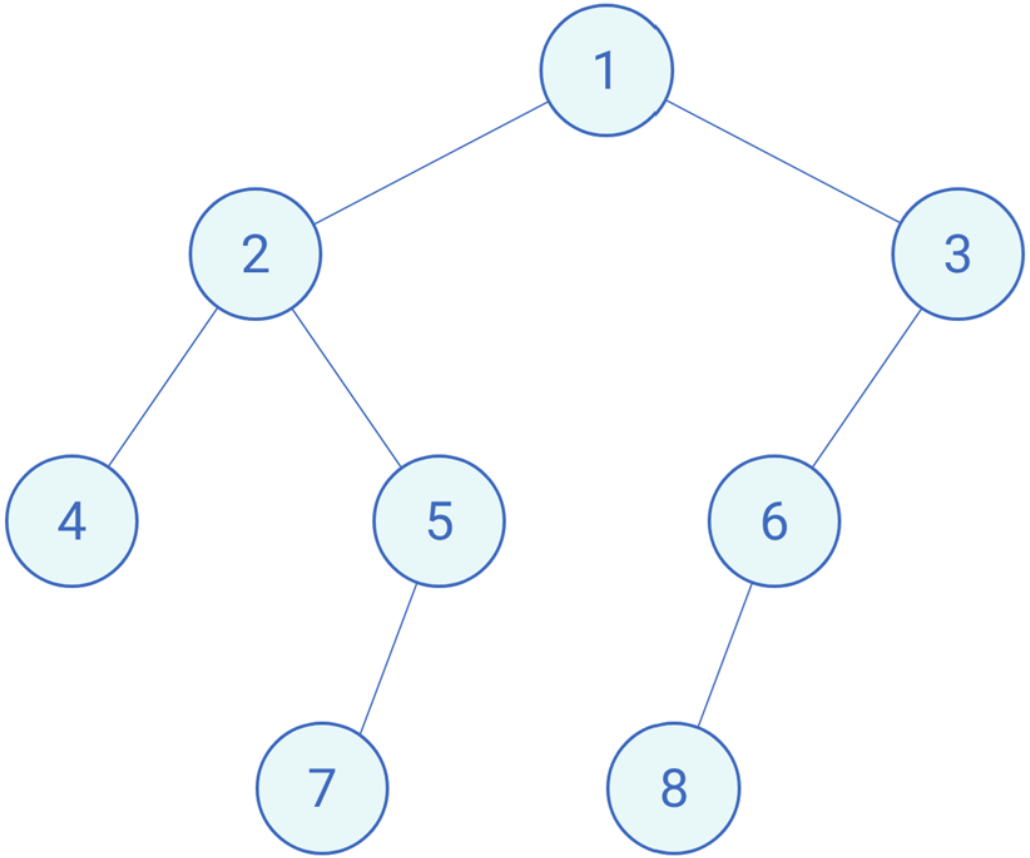


Inorder - B D A G E C H F I.

Preorder - A B D C E G F H I.

Postorder - D B G E H I F C A.

Find Inorder, Preorder and Postorder of below tree



Inorder – 4 2 7 5 1 8 6 3

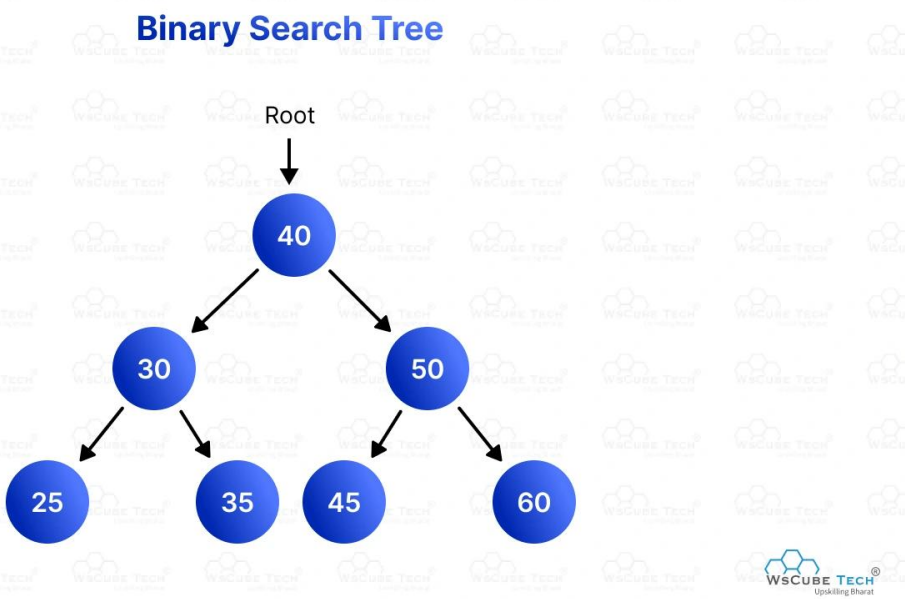
Preorder – 1 2 4 5 7 3 6 8

Postorder – 4 7 5 2 8 6 3 1

Binary Search Tree

A **Binary Search Tree** is a binary tree where:

- Left subtree contains values **less than** the node
- Right subtree contains values **greater than** the node
- No duplicates (common rule)



Advantages Binary Search Tree

1. **Fast Searching**
2. Fast Insertion & Deletion
3. Efficient Searching for Range Queries
4. Easy to Find Min/Max
5. Useful for Hierarchical Data

Construct BST from following

1. **5, 1, 3, 4, 2, 7, 6, 8**
2. M, B, Q, Z, A, C

BST insert node – non recursive


```
def insert(self, key):
    new_node = Node(key)

    if not self.root:
        self.root = new_node
        return

    current = self.root
    parent = None

    while current:
        parent = current
        if key < current.key:
            current = current.left
        else:
            current = current.right
    if key < parent.key:
            parent.left = new_node
        else:
            parent.right = new_node

bst = BST()
for x in [50, 30,70]:
    bst.insert(x)
```

BST Inorder Traversal– non recursive


```
def inorder(self):
    stack = []
    current = self.root

    while True:
        while current:
            stack.append(current)
            current = current.left

        if not stack:
            break


current = stack.pop()
        print(current.key, end=" ")

        current = current.right
```

```
 print("Inorder: ", end="")
bst.inorder()
```

BST Preorder Traversal– non recursive


```
def preorder(self):
    if not self.root:
        return

    stack = [self.root]

    while stack:
        node = stack.pop()
        print(node.key, end=" ")

        if node.right:
            stack.append(node.right)
        if node.left:
            stack.append(node.left)
```

```
print("Preorder: ", end="")
bst.preorder()
```

BST Postorder Traversal– non recursive


```
def postorder(self):
    if not self.root:
        return

    stack1 = [self.root]
    stack2 = []

    while stack1:
        node = stack1.pop()
        stack2.append(node)

        if node.left:
            stack1.append(node.left)
        if node.right:
            stack1.append(node.right)

    while stack2:
        print(stack2.pop().key, end=" ")
```

```
print("postorder: ", end="")
bst.postorder()
```

What is an AVL Tree

An **AVL Tree** is a **self-balancing Binary Search Tree (BST)**.
It maintains the rule:

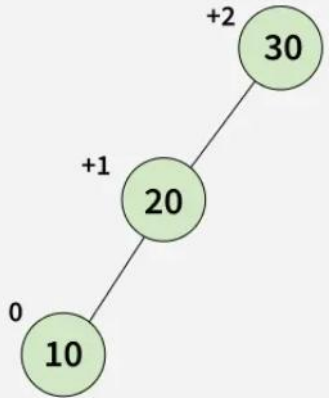
Balance Factor (BF) = height(left subtree) – height(right subtree)

For every node:

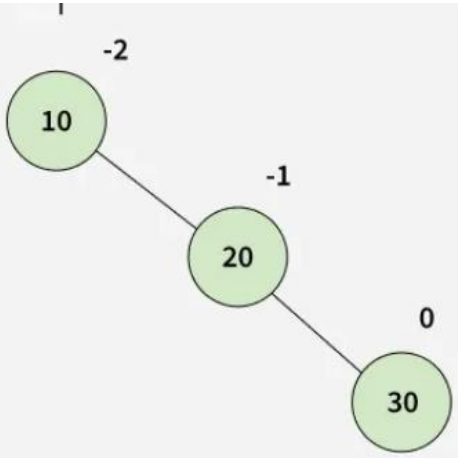
For a Balanced Tree(for every node): -1 ≤ Balance Factor ≤ 1

AVL Tree

Left unbalanced tree



Node '30' has a balance factor of +2, which lies outside the acceptable range of -1 to +1.



Node '10' has a balance factor of -2, which lies outside the acceptable range of -1 to +1.

AVL Tree Rotations

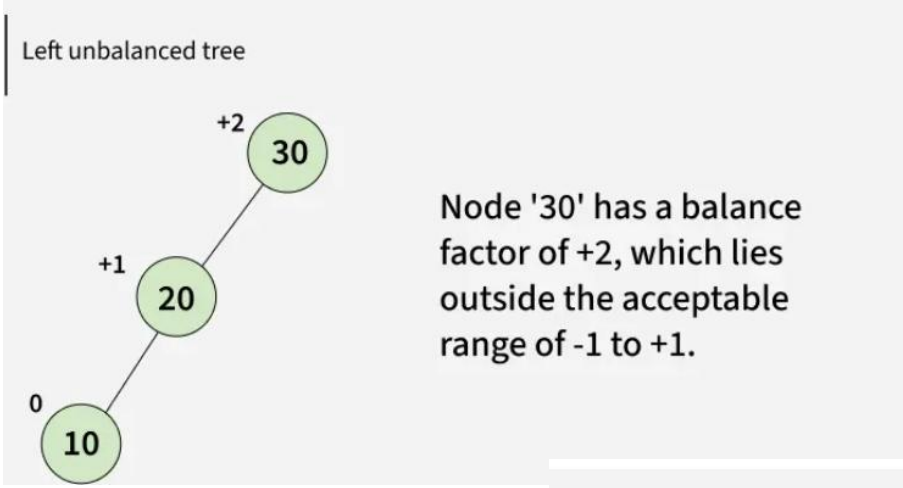
1. LL – Left to Left Rotation
2. RR – Right to Right Rotation
3. LR – Left to Right Rotation
4. RL – Right to Left Rotation

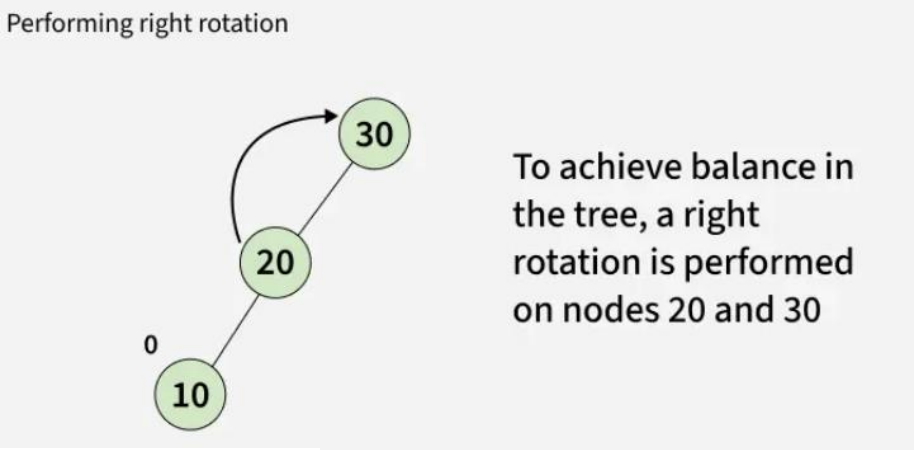
AVL Tree Rotations

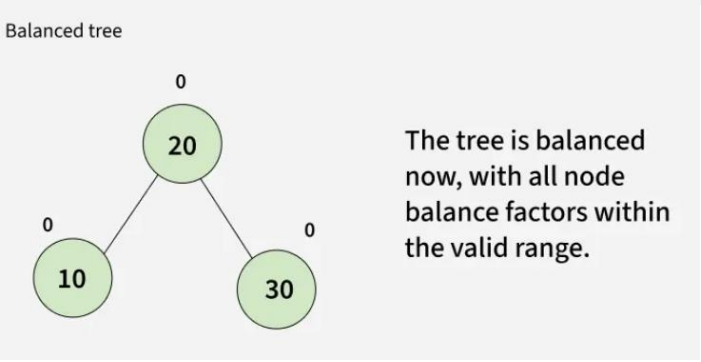
1. LL – Left to Left Rotation :

Occurs when:

- A node becomes **left-heavy** (BF > 1)
- And the new key is inserted into its **left child's left subtree**



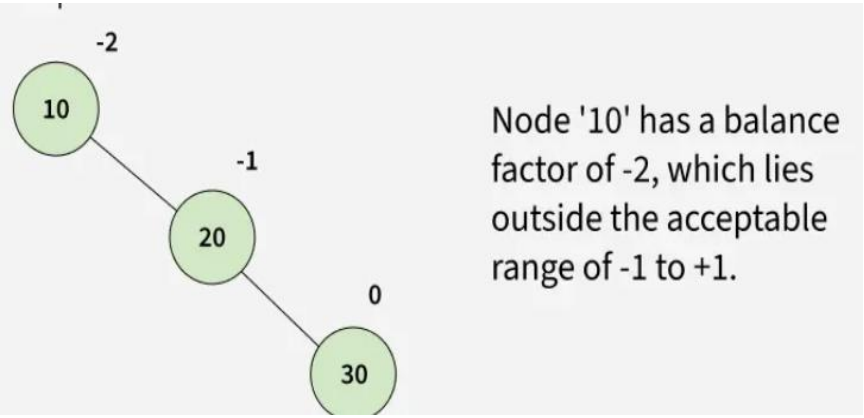


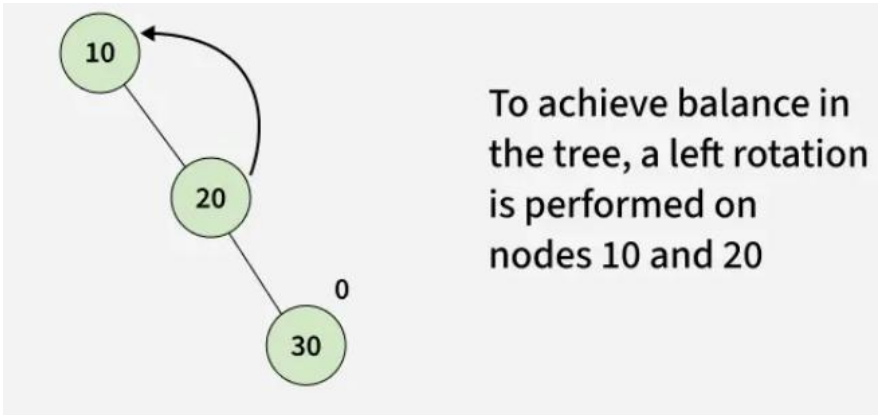


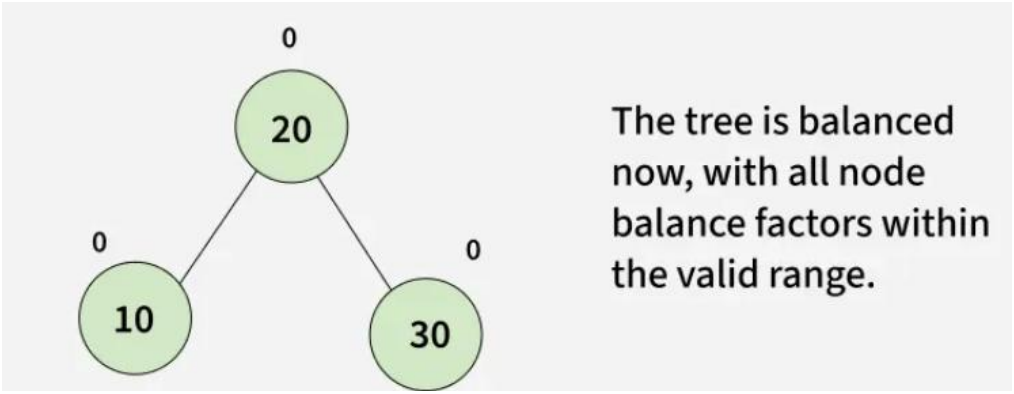
AVL Tree Rotations

1. RR – Right to Right Rotation :

Occurs when a node is inserted into the right subtree of the right child, making the balance factor **less than -1**.



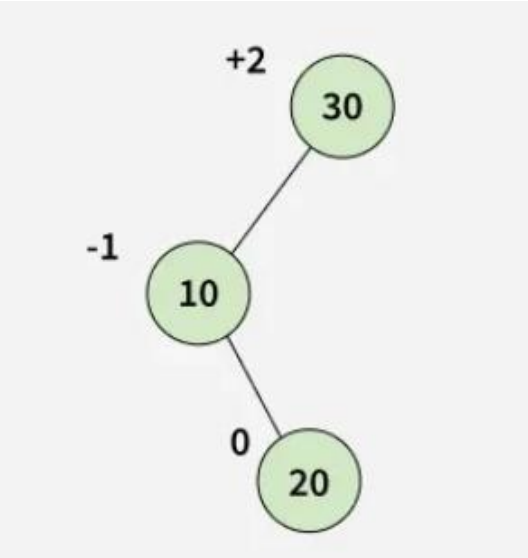


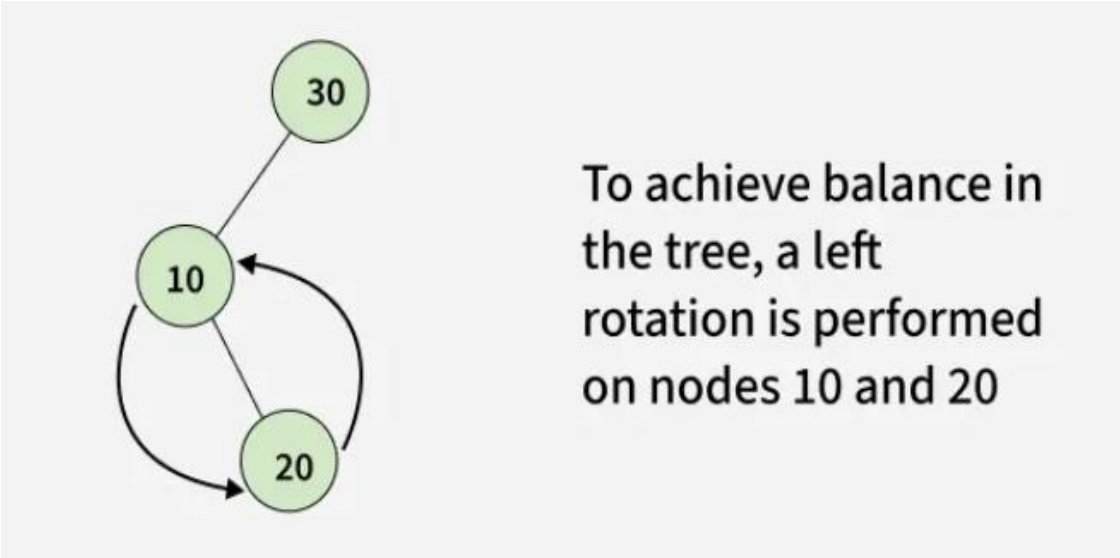


AVL Tree Rotations

1. LR – Left Right Rotation:

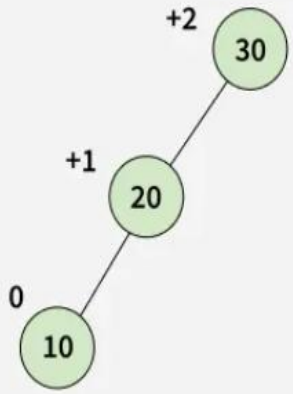
- **Occurs when** a node is inserted into the right subtree of the left child, which disturbs the balance factor of an ancestor node, making it left-heavy.



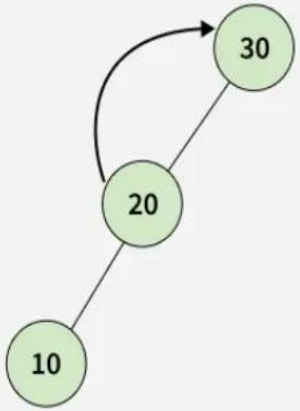


AVL Tree Rotations

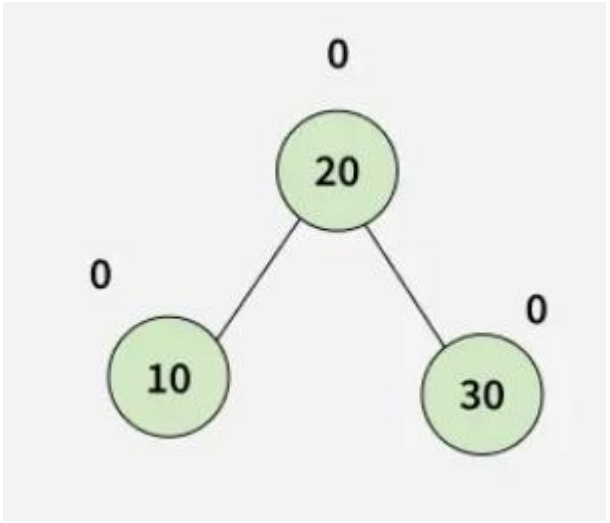
1. LR – Left Right Rotation:



we need to perform right rotation on nodes 30 and 20 to balanced the tree

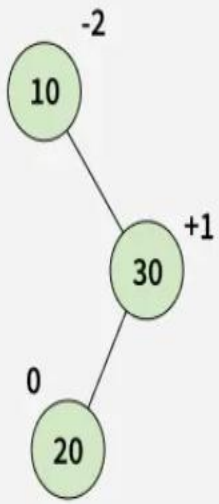


To achieve balance in the tree, a right rotation is performed on nodes 20 and 30

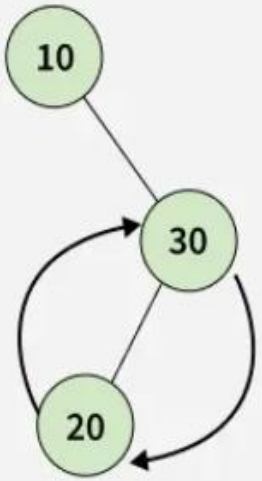


1. RL – Right Left Rotation:

- Occurs when a node is inserted into the left subtree of the right child, which disturbs the balance factor of an ancestor node, making it **right-heavy**.



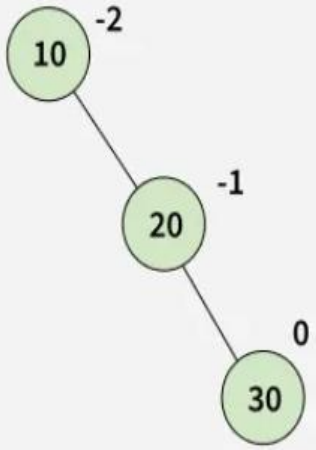
Node '10' has a balance factor of -2, which lies outside the acceptable range of -1 to +1.



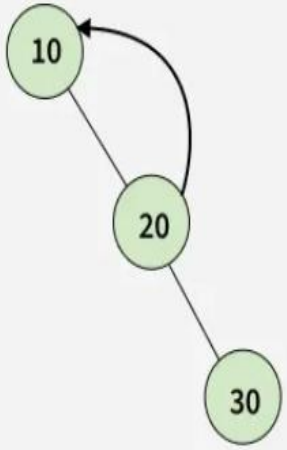
To achieve balance in the tree, a right rotation is performed on nodes 30 and 20

AVL Tree Rotations

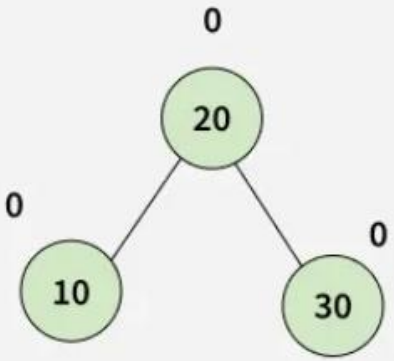
1. RL – Right Left Rotation:



we need to perform left rotation on nodes 10 and 20 to balanced the tree



To achieve balance in the tree, a left rotation is performed on nodes 10 and 20



AVL Tree

1. 3,5,11,8,4,1,12,7,2,6
2. 14,17,11,7,53,4,13,12,8,60,19,16,20
3. 0, 30, 90, 20, 40, 35

Delete Node BST

Depending on the structure of the node to be deleted, there are three possible scenarios:

Case 1: Node has No Children (Leaf Node)

If the target node is a leaf node, it can be directly removed from the tree since it has no child to maintain.

Case 2: Node has One Child(Left or Right Child)

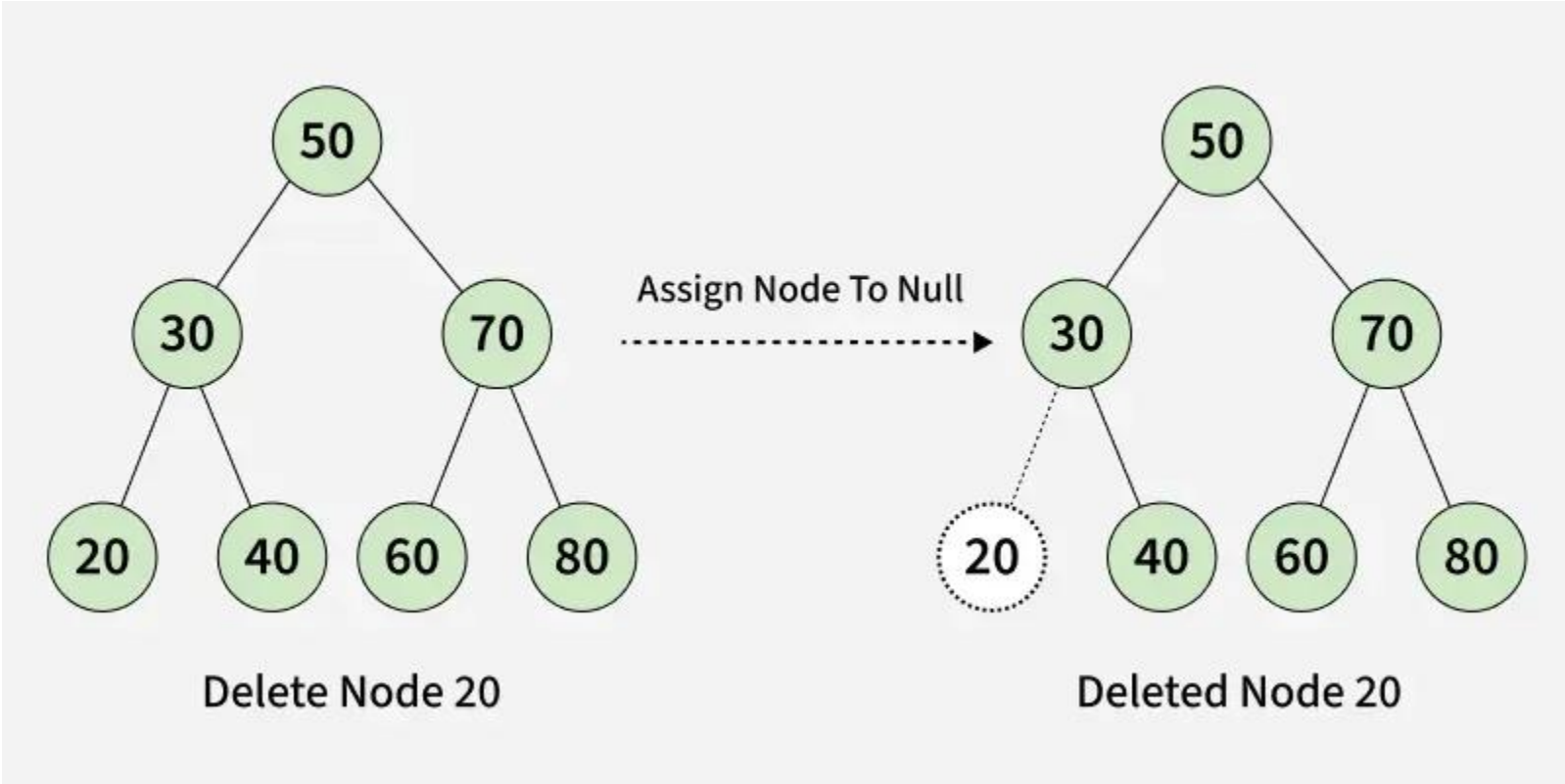
If the target node has only one child, we remove the node and connect its parent directly to its only child.

Case 3: Node has Two Children

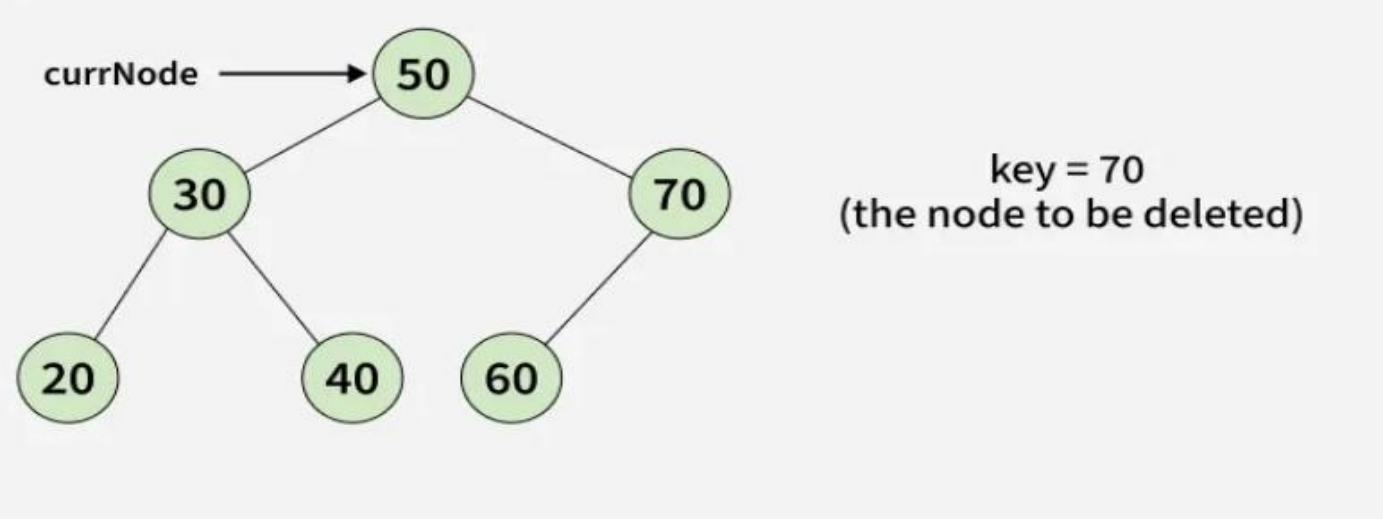
To maintain the BST property, we need to find a replacement node for the target. The replacement can be either:

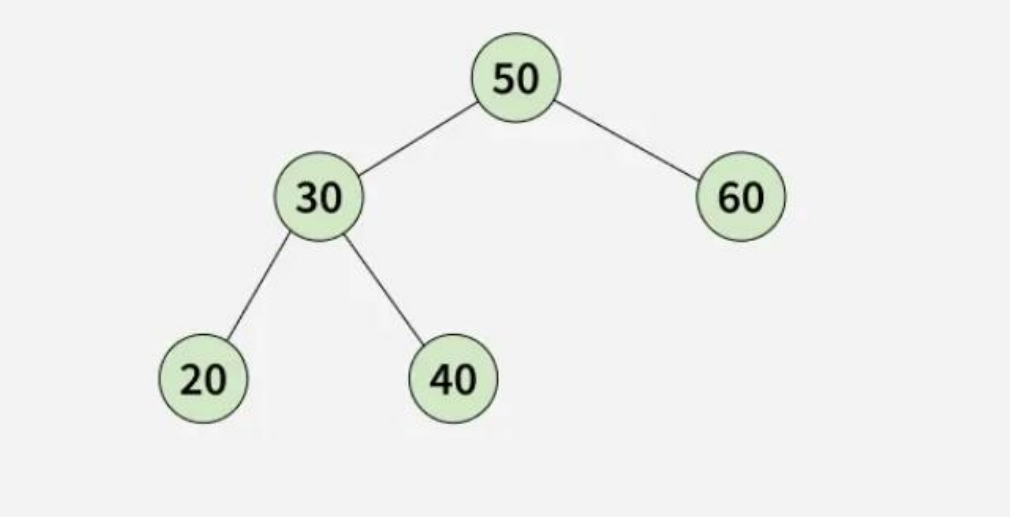
- The inorder successor
- The inorder predecessor

Case 1: Node has No Children (Leaf Node)

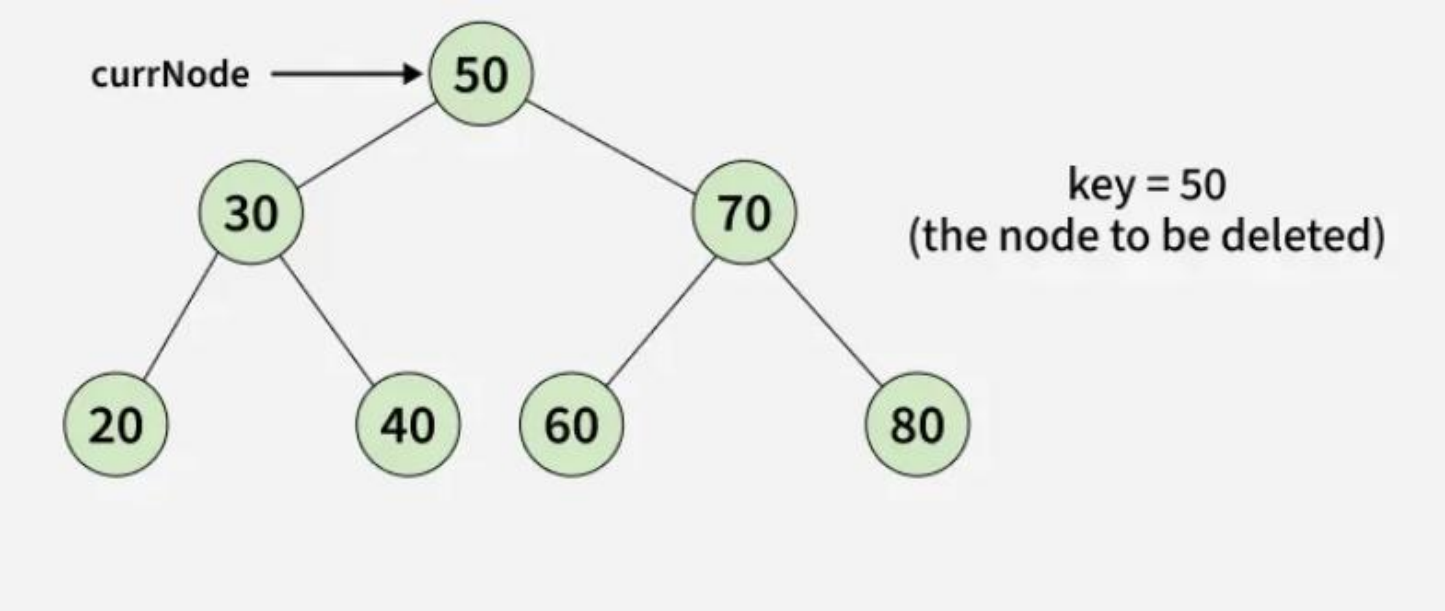


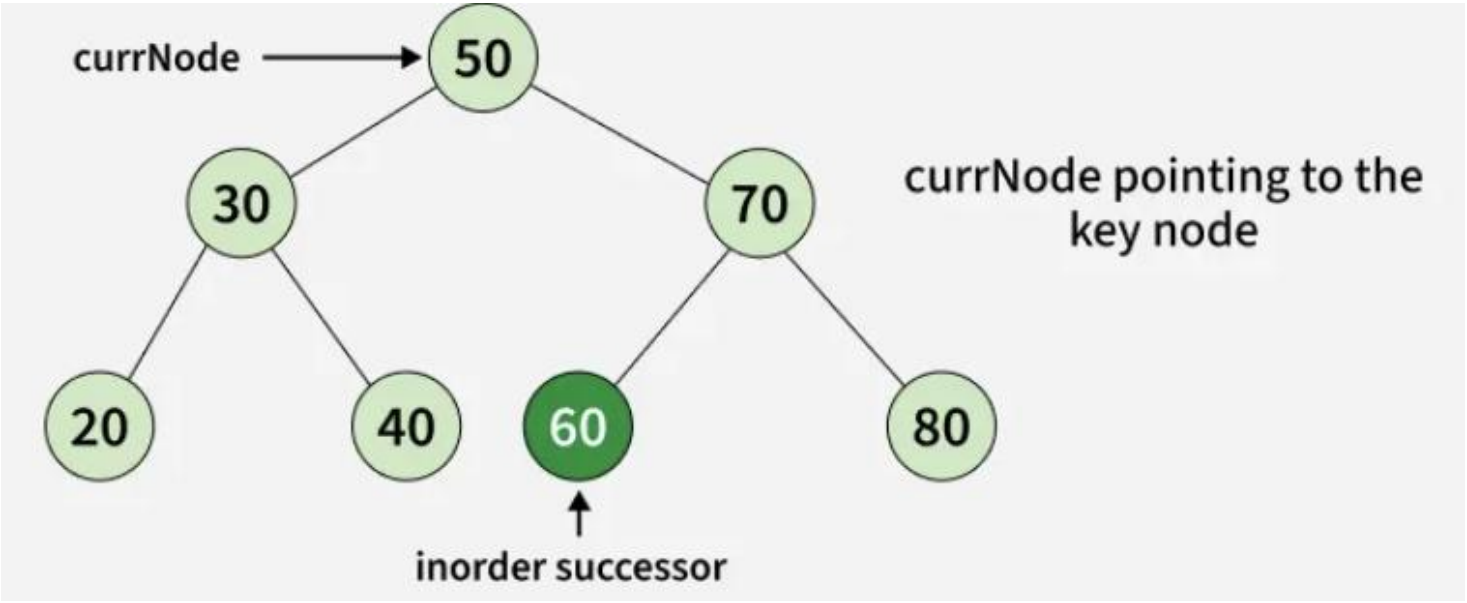
Case 2: Node has One Child(Left or Right Child)



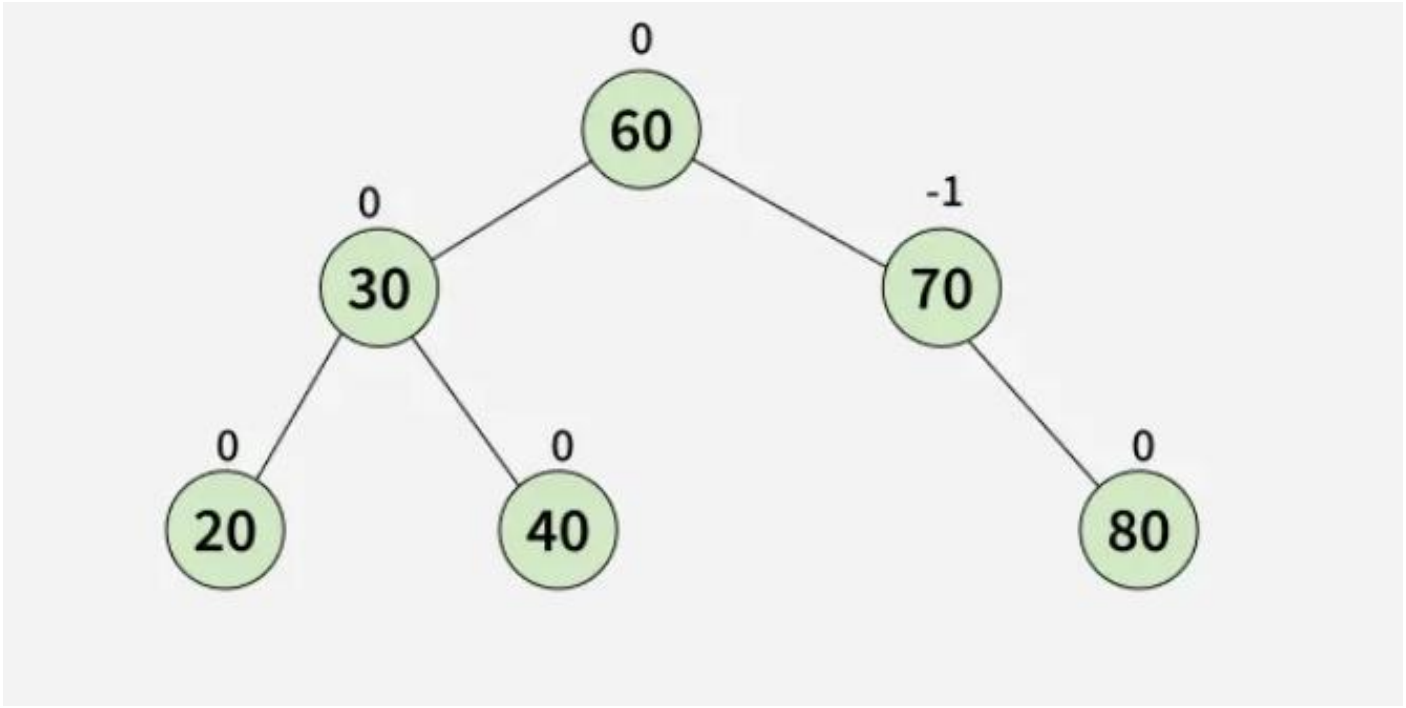


Case 3: Node has Two Children



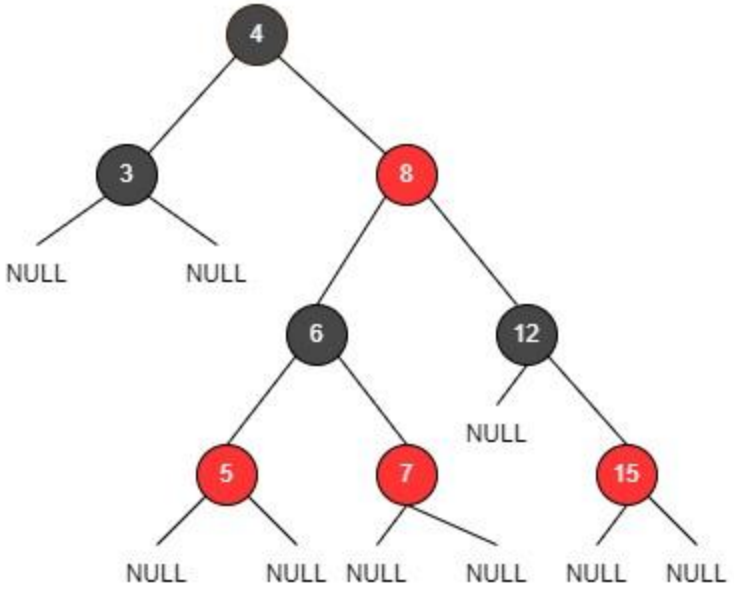


Final BST



Red-Black Tree

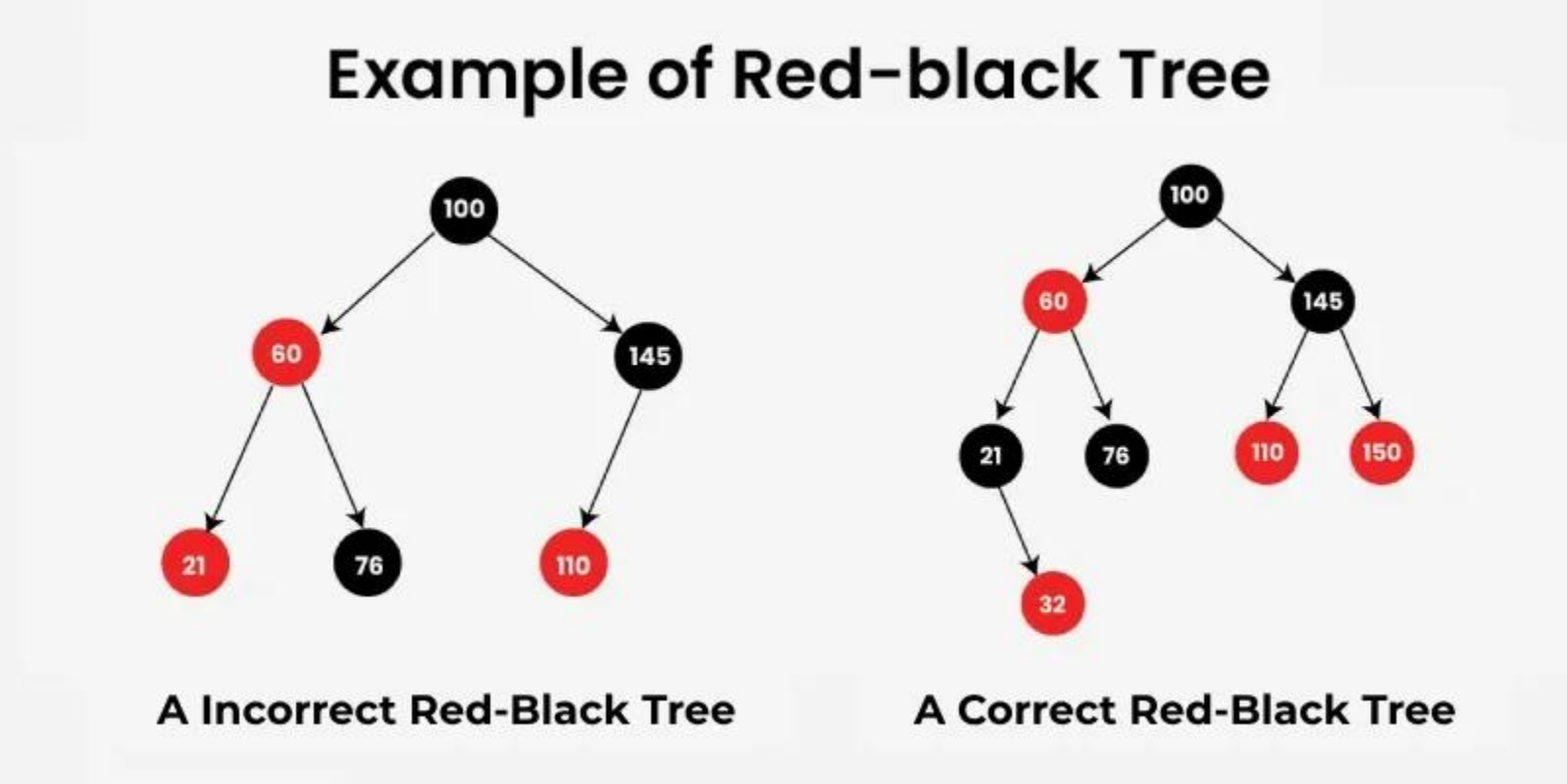
- Red-Black Trees are another type of the Balanced Binary Search Trees with two colored nodes: Red and Black.
- The colors are used to maintain balance during insertions and deletions, ensuring efficient data retrieval and manipulation.



Properties of Red-Black Trees

- **Node Color:** Each node is either red or black.
- **Root Property**: The root of the tree is always black.
- **Red Property**: Red nodes cannot have red children (no two consecutive red nodes on any path).
- **Black Property**: Every path from a node to its descendant null nodes (leaves) has the same number of black nodes.
- **Leaf Property**: All leaves (NIL nodes) are black.

Properties of Red-Black Trees



Insertion in Red Black Tree

Case 1 : Root color is always black

Case 2 : Red- Red Violation:

- If the new node's parent is Red :

  - **Case 2.1: Uncle is Red**: Recolor the parent and uncle to **black**, and the grandparent to **red**. Then move up the tree to check for further violations.
  - **Case 2.2 : Uncle is Black or Null** :
    Perform necessary rotations to balance the tree and recolors the nodes accordingly

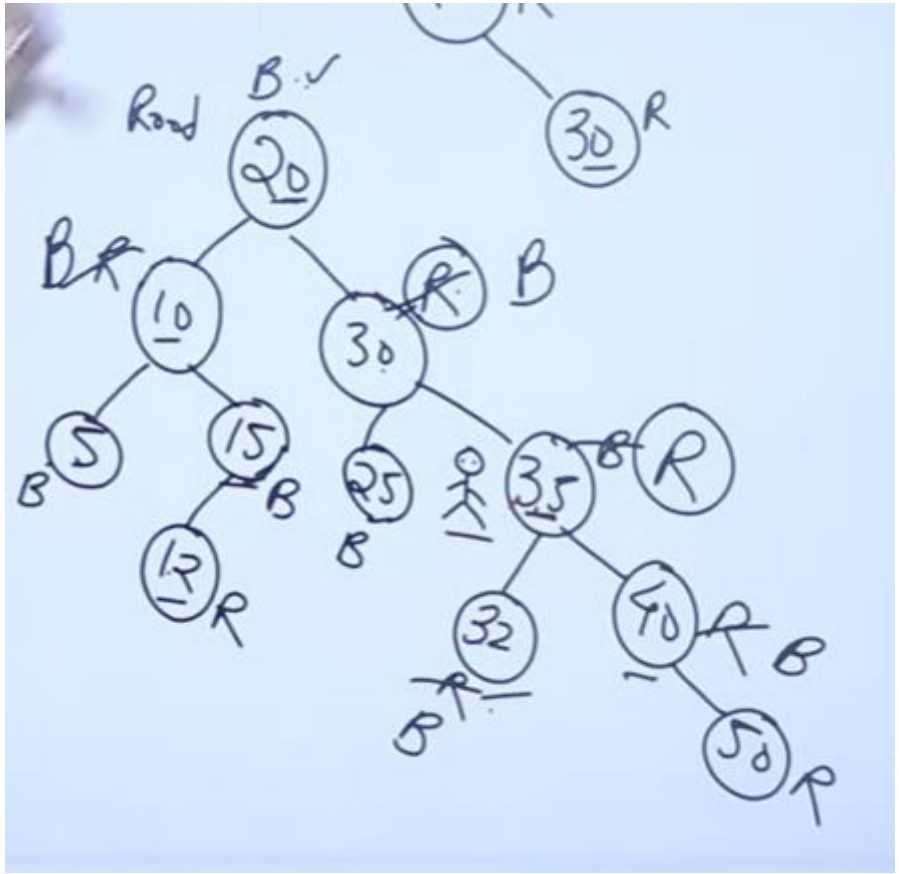
Properties of Red-Black Trees

- **Node Color:** Each node is either red or black.
- **Root Property**: The root of the tree is always black.
- **Red Property**: Red nodes cannot have red children (no two consecutive red nodes on any path).
- **Black Property**: Every path from a node to its descendant null nodes (leaves) has the same number of black nodes.
- **Leaf Property**: All leaves (NIL nodes) are black.

Solve Below Example

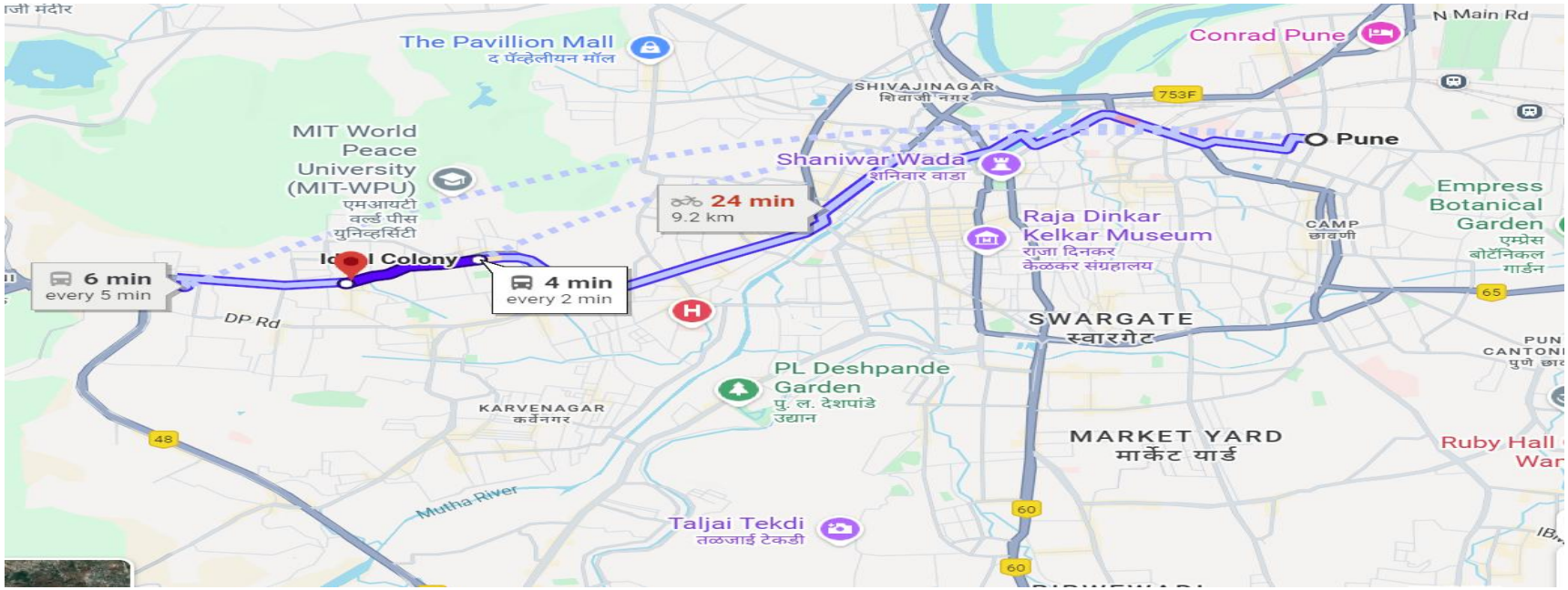
- 10, 20, 30, 15, 25, 5, 12, 35, 40, 32, 50
- 18, 8 , 7 , 15, 16, 30, 25, 40, 60, 2, 1, 70

Solve Below Example



Graph Data Structure

- A graph is a non-linear data structure made up of vertices (nodes) and edges (connections) that represent relationships between objects.
- Example: On a map, each city is a vertex, and each road connecting two cities is an edge. This way, a graph represents how cities are linked.



Components Graph Data Structure

Vertices: vertices are also known as vertex or nodes**.**

Edges: Edges are drawn or used to connect two nodes of the graph. It can be ordered pair of nodes in a directed graph. Edges can connect any two nodes in any possible way. There are no rules.

Adjacency – Two node or vertices are adjacent if they are connected to each other through an edge.

Path – Path represents a sequence of edges between the two vertices.

Applications of Graph

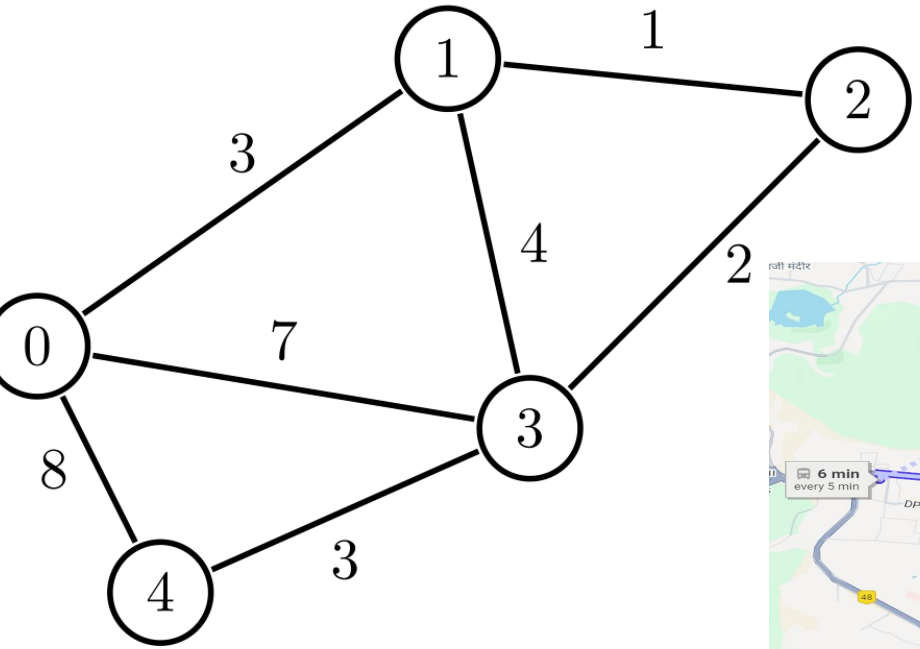
- **Social networks :** Represents relationship with users
- **Google Maps** : Find out the best route
- **Network Routing** : Optimizing data flow in communication
- **Game Development**
- **Blockchain & Cryptography**

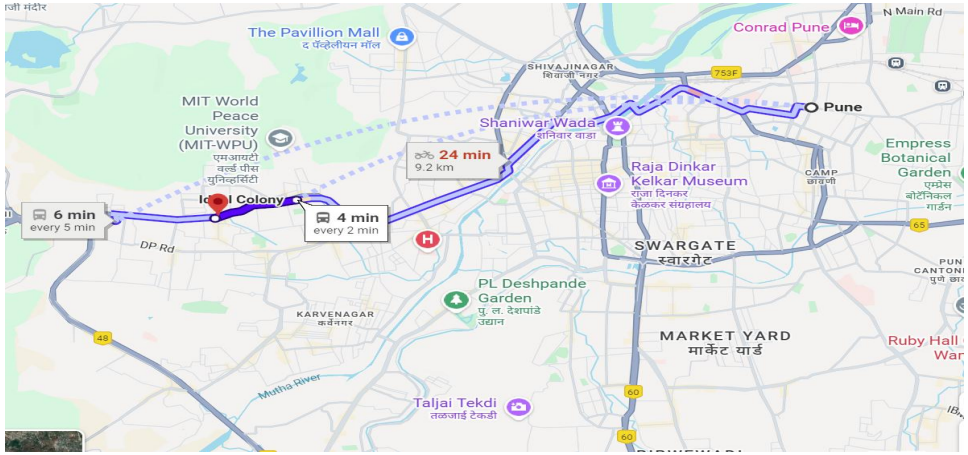
Types of Graph

Based on weight :

- **Weighted Graphs**

A **weighted graph** is a graph where each edge has a number (weight) that represents distance, cost, or time.

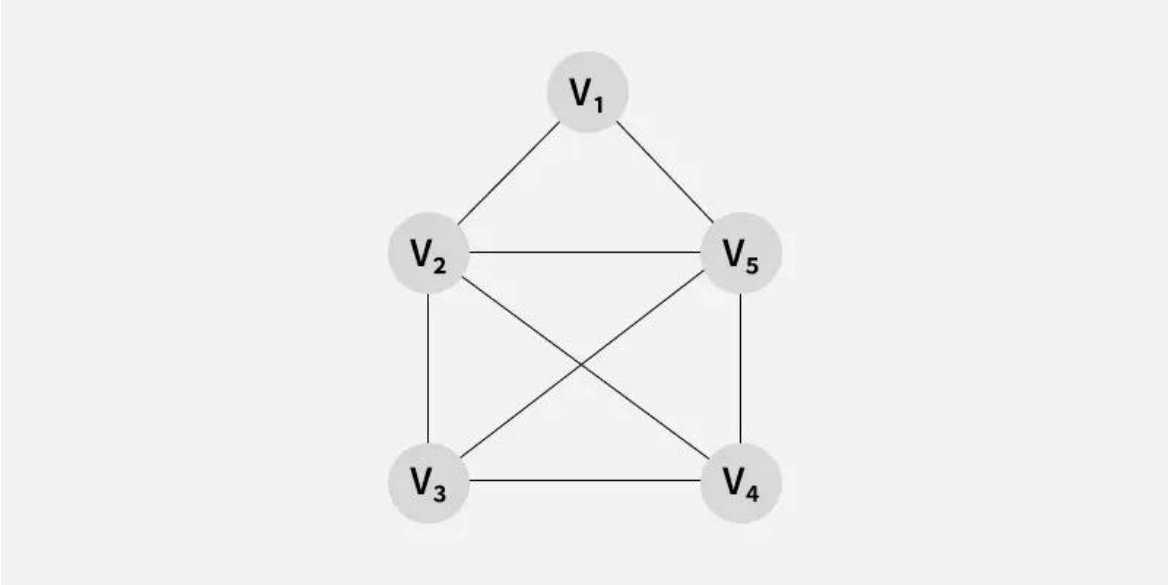




Types of Graph

Unweighted Graphs

- An unweighted graph is a graph where all edges are treated equally, with no extra values like distance or cost.



Types of Graph

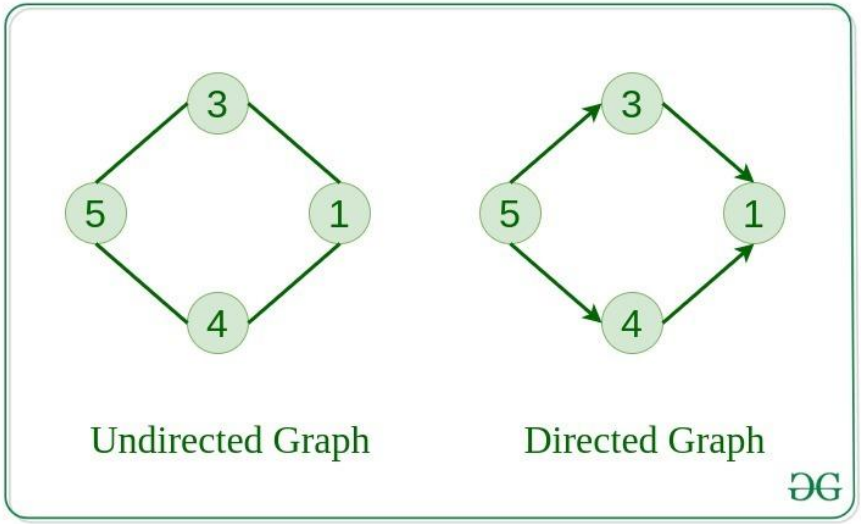
Based on Edge Direction:

Undirected Graph

A graph in which edges do not have any direction. That is the nodes are unordered pairs in the definition of every edge.

Directed Graph

A graph in which edge has direction. That is the nodes are ordered pairs in the definition of every edge.



Representation of Graph Data Structure:

There are multiple ways to store a graph: The following are the most common representations.

- **Adjacency Matrix :**

For a graph with V vertices, the adjacency matrix is a V × V

matrix: matrix[i][j] = 1 → edge exists between vertex i and j

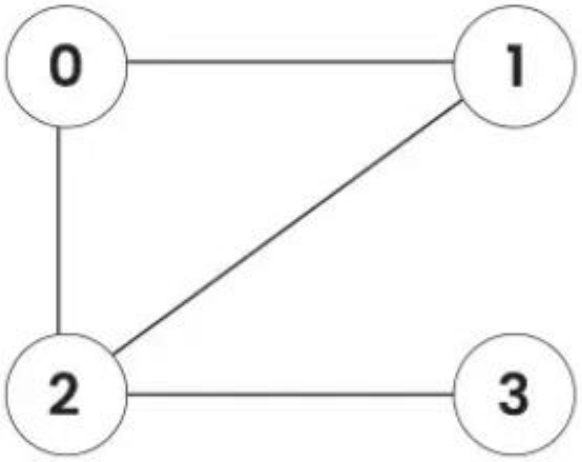
matrix[i][j] = 0 → no edge For weighted graphs, store the weight

instead of 1.

Representation of Graph Data Structure:

- **Adjacency Matrix :**

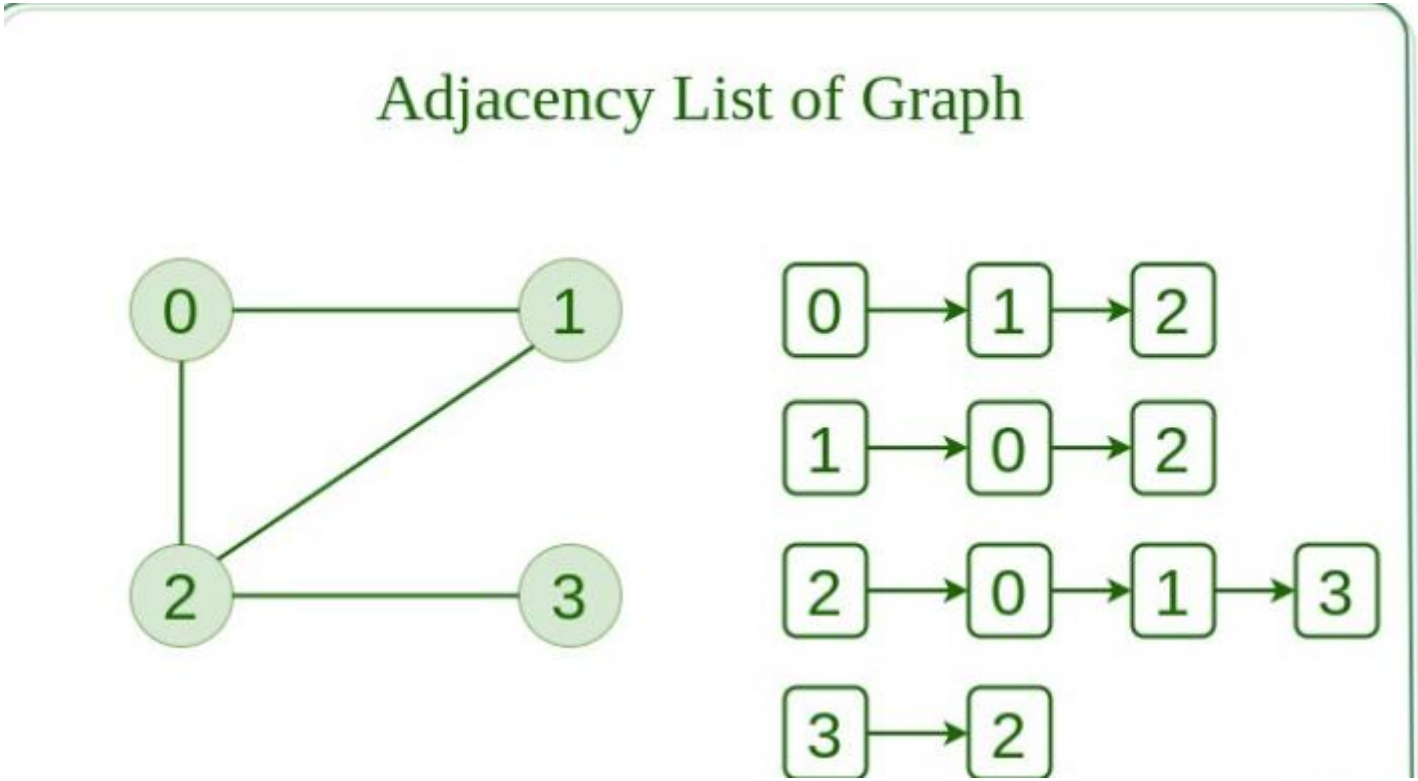
Adjacency Matrix of Graph



	0	1	2	3
0	0	1	1	0
1	1	0	1	0
2	1	1	0	1
3	0	0	1	0

Representation of Graph Data Structure:

- **Adjacency List :** This graph is represented as a collection of array lists. There is an array of pointer which points to the edges connected to that vertex.



Breadth First Search (BFS) :

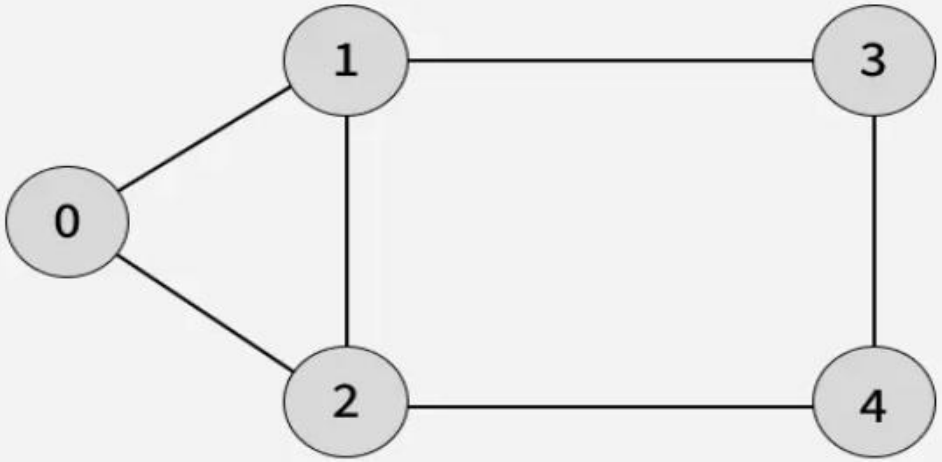
Breadth First Search (BFS) is a graph traversal algorithm that starts from a source node and explores the graph level by level. First, it visits all nodes directly adjacent to the source. Then, it moves on to visit the adjacent nodes of those nodes, and this process continues until all reachable nodes are visited.

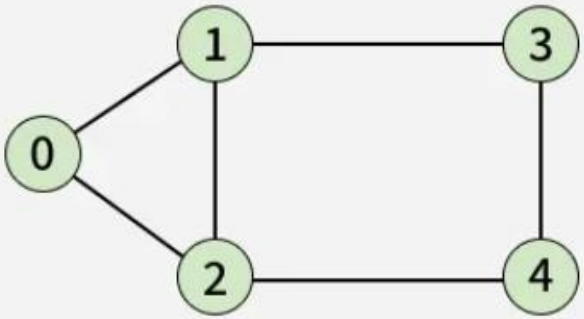
Breadth First Search (BFS) :

1. **Mark the start node as visited** and place it in a queue.
2. **Remove a node from the queue**, visit it, and perform necessary actions.
3. **Add all unvisited neighboring nodes** of the current node to the queue and mark them as visited.
4. **Repeat steps 2 and 3** until the queue is empty, indicating that all reachable nodes have been processed.

Traversal Technique of Graph:

Given the following graph, find its BFS traversal





visited[]=	T	T	T	T	T
res[]=	0	1	2	3	4
queue:					

BFS traversal order is 0, 1, 2, 3, 4.

Depth First Search (DFS) :

- Depth First Search (DFS) starts from a given source vertex and explores one path as deeply as possible. When it reaches a vertex with no unvisited neighbors, it backtracks to the previous vertex to explore other unvisited paths. This continues until all vertices reachable from the source are visited.

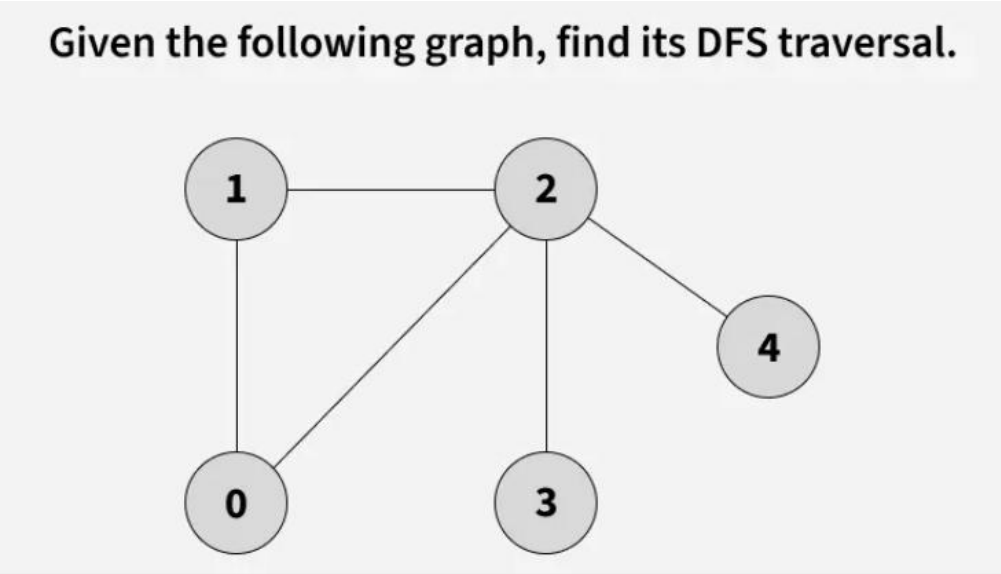
Depth First Search (DFS) :

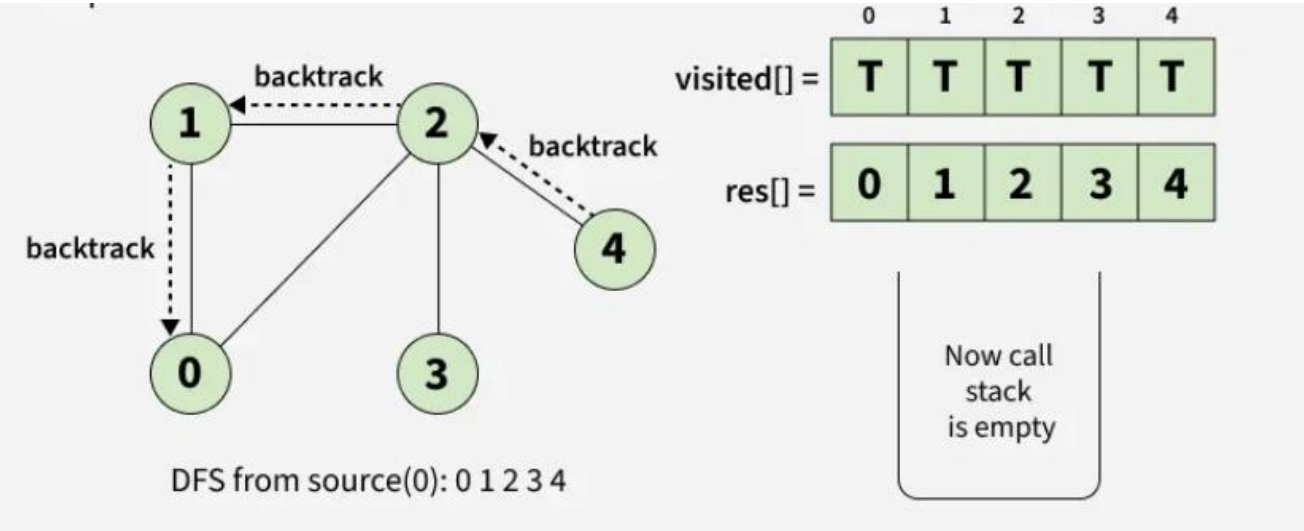
- Depth First Search (DFS) starts from a given source vertex and explores one path as deeply as possible. When it reaches a vertex with no unvisited neighbors, it backtracks to the previous vertex to explore other unvisited paths. This continues until all vertices reachable from the source are visited.

Depth First Search (DFS) :

- Start with an empty stack and an empty visited list.
- Push the initial vertex (starting vertex) onto the stack.
- While the stack is not empty:
  - Pop the top vertex from the stack.
  - If the vertex has not been visited, add it to the visited list.
  - Push all unvisited adjacent vertices of the current vertex onto the stack.
- The process continues until the stack is empty.

Traversal Technique of Graph:





Find BFS and DFS of following Graph

